

ZIG –

Space Museum: Implementation Roadmap and Agent Brief

New feature in the existing ZIG app | Lab-screen entry | Kotlin + Jetpack Compose | Google Filament
Continuous gated execution M-1 to M12. Build spec revision 3.0

SPACE MUSEUM - IMPLEMENTATION ROADMAP AND AGENT BRIEF

Written FOR AN AI CODING AGENT working inside the EXISTING ZIG repository: the integration rule, the pinned toolchain, the repository layout, the technical specification, per-object requirements, the best available asset per object, milestones M-1 to M12, continuous gated execution, verification with no emulator and no phone, the prompt library, testing, reporting, risks and GitHub discipline.

THE FEATURE: SPACE MUSEUM, reached from one new card in the existing Lab screen.

Thirteen objects, one at a time, inspect anywhere, fully offline, no network permission.

Sun | Mercury | Venus | Earth | Moon | Mars | Jupiter | Saturn | Uranus | Neptune | Milky Way | ISS | Black hole

THE THREE RULES THAT SHAPE THIS REVISION

1. TOUCH NOTHING IRRELEVANT. This brief does not name files, because its author cannot see your repository. One test governs every edit: does Space Museum fail to work without this change? If not, do not make it. ZIG's own calculations, screens, models, strings, assets, tests and build configuration stay exactly as they are (section 26).
2. LICENSING IS NOT THE AGENT'S JOB. The owner handles licences, permissions and registries himself, with a law professional, after the feature is complete. Choose the best asset per object on quality alone, and record product, publisher and URL for every one of them so the owner can clear rights afterwards (section 27).
3. PUSH RARELY. One push per completed milestone. If GitHub looks disconnected it is throttling: wait 15 minutes and retry, up to four times, and keep building meanwhile (section 28).

EXECUTION: CONTINUOUS AND GATED (section 23). Work M-1 to M12 in order without waiting for a human; when the green gate (23.2) and the DoD pass, commit, tag, update PROGRESS.md and start the next. Stop only for the hard stop conditions. Device-only evidence never blocks progress.

NO EMULATOR, NO PHONE: four tiers (section 24) - host/CI, a CI emulator on KVM, Firebase Test Lab's free quota, and the owner's phone. Appendix H has the workflows; Appendix I has the card sketch.

THE KICKOFF PROMPT AT THE TOP IS WHAT YOU PASTE TO THE AGENT, WITH THIS PDF ATTACHED.

=====

ZIG – SPACE MUSEUM: IMPLEMENTATION ROADMAP AND AGENT BRIEF New feature inside the existing ZIG app | Lab-screen entry point | Kotlin + Compose | Filament

Build specification revision 3.0 – 2026-09-14

Companion document to: ZIG_NASA_EYES_LEVEL_RESEARCH (revision 3)

Revision 3.0 supersedes revision 2.0. Everything below is written against it:

- (a) LICENSING IS OUT OF SCOPE. The owner handles all licensing, attribution rights and permissions himself, with a law professional, after the feature is finished and he is satisfied with it. The agent makes no licence decisions, keeps no licence registry, and is never blocked by a licence question. Asset selection is a pure quality decision (section 27).
 - (b) FILE-LEVEL INSTRUCTIONS ARE GONE. The agent knows the repository; the author of this brief does not. The rule is no longer "do not touch these files" but "do not touch anything irrelevant" (section 26).
 - (c) GITHUB. Section 28 is binding: push discipline, and what to do when GitHub looks disconnected when it is only throttling.
- =====

KICKOFF PROMPT – COPY THIS TO YOUR AGENT TOGETHER WITH THIS PDF

You are adding a new feature called SPACE MUSEUM to ZIG, an existing Android astronomy app written in Kotlin with Jetpack Compose. The feature renders one high-fidelity object at a time - thirteen of them, from the Sun to a black hole - fully offline, using Google Filament. The attached PDF "Space_Museum_Roadmap" is your specification. A second PDF, the NASA-Eyes-level rendering research, is background: use it only where the brief points at it.

RECONNAISSANCE (before writing any code)

1. Read the repository: build files, module graph, version catalog, navigation, theming, resources, existing features, tests, CI.
2. Find the Lab screen and read it end to end. How a card is added, how a tap navigates, what the existing cards look like - that comes from the repository, not from the brief. The repository wins wherever the two disagree. Record every difference in docs/decisions/DECISIONS.md.
3. Before any production code, write docs/SPEC_MUSEUM.md (the feature as it will exist in this repository) and docs/integration/CHANGED_FILES.md (every pre-existing file you expect to touch, one line each, with the reason). Never write a path or symbol you have not read.

DO NOT TOUCH ANYTHING IRRELEVANT This brief cannot tell you which files matter: you know the repository, its author does not. So the rule is not a list of forbidden files - it is a test applied to every single edit: DOES SPACE MUSEUM FAIL TO WORK WITHOUT THIS CHANGE? If the answer is not an unqualified yes, do not make the change. No refactoring, no renaming, no reformatting, no dependency bumps, no version alignment, no "while I was here" fixes. Existing calculations (including everything to do with the ISS), screens, models, strings, assets, tests and build configuration stay exactly as they are. Bugs you notice elsewhere: record them in DECISIONS.md under "found, not touched" with file and line, and leave them alone. If the feature cannot be added without a substantial rework of existing code, stop and report it.

LICENSING IS NOT YOUR PROBLEM Do not spend a single step on licences, attribution rights, permissions, registries or legal questions. The owner handles all of that himself, with a law professional, after the feature is finished and he is satisfied with it. Choose the best asset for each object on quality alone (section 27). What you must do is record provenance: for every asset that ships, the product name, the publisher and the URL it came from, in the pack manifest, so the owner can trace and clear each one afterwards. And write the feature's code yourself: third-party source code is never pasted or vendored into this repository - external code enters only as the pinned dependencies of section 3.

HOW TO USE THE BRIEF

4. Read sections 1-4, 26 (integration and the no-irrelevant-edits rule), 23 (continuous execution and the green gate), 27 (choosing assets) and 28 (GitHub discipline) before coding. These override anything that conflicts.
5. Then work the milestone plan in section 17 CONTINUOUSLY, starting at M-1: implement a milestone, satisfy the green gate (23.2) and that milestone's Definition of Done, commit, update PROGRESS.md, and start the next milestone immediately. Do not wait for a human between milestones and do not ask for

approval.

6. Read only the specification sections a milestone cites; do not read the whole document up front.
7. Stop only for the hard stop conditions in section 23.4. When you stop, state the milestone, the exact blocker, the evidence, what you tried, and the options.

GITHUB

8. Push rarely and deliberately (28.2): one push per completed milestone, batched commits, no per-file commits, no two pushes within 20 minutes, no polling loops against the API. Budget roughly one push per milestone, not one per task.
9. If GitHub appears disconnected, unauthenticated, or refuses a push, it is almost always temporary throttling. Do not re-authenticate, do not re-clone, do not create a repository, do not force-push. Wait 15 minutes and retry the same command, up to four times over about an hour. A sync failure is never a hard stop: keep building locally, note it in PROGRESS.md, and retry at the next milestone boundary.

WORKING METHOD

10. Additive by default: new modules, new packages, new files. Prefer new code to modified code, and additive changes to rewrites.
11. Keep the module rules of 4.2: the engine owns every Filament type, the model is pure Kotlin, features talk to the engine through interfaces, offline tools are JVM-only and never ship.
12. Reuse the app's existing design system, typography, resources and conventions, so Space Museum looks like it always belonged there.
13. The ISS visual model is a standalone inspectable object. It must never replace, feed or alter ZIG's existing ISS calculations, or any other existing subsystem.
14. Record every non-obvious decision in docs/decisions/DECISIONS.md with the reason and, where relevant, the measurement.
15. Run the gate (23.2) before advancing; report in the section 20 format with numbers, not adjectives. Every claim needs evidence you produced.
16. Prove minimum diff every milestone: ZIG's full test suite still passes unmodified, and docs/integration/CHANGED_FILES.md accounts for every pre-existing file touched.
17. When the plan completes, or on a hard stop, produce the handover report in section 25.

Begin with M-1 and continue until the end of the plan or a hard stop.

HOW TO USE THIS DOCUMENT (READ FIRST)

This document is written FOR AN AI CODING AGENT. It is a build specification, not a research paper. Every instruction is imperative and, wherever possible, verifiable.

Read order for a fresh agent session:

0. THE REPOSITORY FIRST: read the existing ZIG app before reading this document. Where the repository and this brief disagree (module names, package roots, Compose style, dependency versions), THE REPOSITORY WINS. Record every difference in docs/decisions/DECISIONS.md.
1. Part I completely (scope, rules, versions, repository layout). Never skip.
2. Section 26 (integration: do not touch anything irrelevant), section 23 (continuous execution, the green gate, the hard stop list), section 27 (choosing the best asset per object), section 28 (GitHub discipline and rate limits), and section 24 (verification with no local emulator or phone).
3. Then work the milestone plan in section 17 from the first milestone that is not yet green, reading only the specification sections that milestone cites. Do not read the whole document before starting work; it is large on purpose

and reading it all is a waste of context.

The rules that override everything else:

- RULE 1. Work through the milestones IN ORDER, CONTINUOUSLY, WITHOUT WAITING FOR A HUMAN BETWEEN THEM. After each milestone, run its gate (section 23.2). Gate green -> commit, tag, update PROGRESS.md, and begin the next milestone in the same working session. The only reasons to stop are the hard stop conditions in section 23.4. A GitHub rate limit or authentication error is NOT a hard stop: see section 28.4.
- RULE 2. If a required input is missing, ambiguous, unavailable, or you cannot verify it, STOP and report. Never substitute an unverified asset, URL, library or number.
- RULE 3. Never invent: assets, textures, dataset URLs, product names, API names, or scientific values. Verify against the pinned dependency or the primary source; if you cannot, say so.
- RULE 4. The app must work with no network. No INTERNET permission in the release manifest. No analytics, telemetry, crash uploaders or ads.
- RULE 5. TOUCH NOTHING IRRELEVANT. Apply one test to every edit: does Space Museum fail to work without this change? If the answer is not an unqualified yes, do not make it (§26.2, §26.7). No refactoring, no renaming, no reformatting, no dependency bumps to satisfy preferences. Existing ZIG code, including every calculation, stays as it is.
- RULE 6. Never claim a measurement you did not make. Device performance, thermals and driver-specific rendering can only be asserted from a real device (sections 24.4-24.7). Everything else is labelled TARGET or ESTIMATE.
- RULE 7. Licensing is not your task. Make no licence decisions, keep no licence registry, and never stop work for a licence question: the owner handles licensing separately, after delivery, with legal counsel (§2.3, §27). Do record provenance (product, publisher, URL) for every asset you ship, and never paste or vendor third-party source code into the repository.

Terminology used throughout:

- LAB SCREEN the existing ZIG screen that will host the Space Museum entry card. You find it in the repository; this brief does not name its path.
- MUSEUM the new feature: the card, the grid of thirteen objects, and the viewer that renders one of them.
- OBJECT one of the thirteen inspectable bodies (Sun, Mercury, Venus, Earth, Moon, Mars, Jupiter, Saturn, Uranus, Neptune, Milky Way, ISS, Black hole).
- PACK one Play Asset Delivery asset pack that upgrades one object.
- TILE STORE the on-device tile pyramid + LRU cache + prefetch system (§7).
- ASSET KIT the offline JVM/CLI toolchain that builds packs from source data (§8). It is part of this repository and it is not optional.
- CEILING the finest real ground resolution available for an object; the point past which the app must label its output as procedural (§14).

PART I – BRIEF

1. MISSION AND SCOPE LOCK

1.1 WHAT IS BEING BUILT

A new feature, SPACE MUSEUM, inside the existing ZIG Android astronomy app, written in Kotlin with Jetpack Compose. It is reached from a new card in the app's existing Lab screen. The feature presents thirteen objects; selecting one shows a single, high-fidelity, locally rendered 3D representation of that object, which the user can orbit, zoom into and inspect from any angle. Exactly one object is ever on screen. The feature is fully offline after installation: no network calls, no streaming, no remote rendering, no WebView. Nothing else in ZIG changes: the feature is additive, and the rest of the app is frozen (section 26).

1.2 WHAT IS EXPLICITLY OUT OF SCOPE (DO NOT BUILD THESE)

- Orbital mechanics, ephemerides, SPICE, Horizons, trajectory propagation.
- Multi-object scenes, inter-body eclipses, transits, occlusions between bodies.
- A simulation clock, timeline scrubbing, or any date arithmetic.
- Mission timelines, spacecraft (other than the ISS as a static model), rovers.
- Any cloud/remote rendering, any WebView, any web page as the renderer.
- Any network access in the runtime application at all.
- Multiple objects rendered simultaneously, in any form, including previews.
- Any modification to existing ZIG behaviour. Not one calculation, screen, string, dependency, permission or asset outside the integration points named in section 26.3.

1.3 THE THIRTEEN OBJECTS (the button list)

1 Sun	2 Mercury	3 Venus	4 Earth
5 Moon	6 Mars	7 Jupiter	8 Saturn
9 Uranus	10 Neptune	11 Milky Way	12 ISS
13 Black hole			

1.4 THE PRODUCT'S CENTRAL IDEA, WHICH DRIVES EVERY TECHNICAL DECISION

The user can zoom anywhere on an object. Therefore the binding constraint is no longer the GPU; it is the resolution of real, available scientific data. The application must (a) ship the deepest real data available per object, (b) know where that data ends, and (c) tell the user when it starts inventing detail (\$5.9, "data HUD"). This is a headline feature, not a disclaimer.

1.5 THE FIVE THINGS THAT MAKE THIS PRODUCT GOOD (keep them working)

1. Real data, traceable: every texture traceable to a named source product.
2. Depth: 0.5 m/px on the Moon and Mars, 15 m/px on Earth, 166 m/px on Mercury, effectively unlimited on the black hole.
3. Honest labelling: true colour vs enhanced vs radar vs simulated vs procedural-beyond-data, all stated in the UI.
4. Physical lighting: the sun is a real illuminant, scaled by real distance, with a user-controlled direction and an exposure control.
5. Restraint: no invented objects, no invented facts, no filler content.

1.6 THE TWO CONSTRAINT CHANGES THAT SHAPE REVISION 2.0

- (a) The feature lives inside ZIG. The user reaches Space Museum from the Lab screen, and ZIG's own functions - including its ISS orbital calculations - keep running untouched. The museum's ISS visual model is an independent object; it must never be wired into those calculations or into any other existing subsystem.
- (b) Asset selection is restricted by nothing except quality (\$27). Licensing, attribution rights and permissions are handled by the owner, separately, after the feature is complete; the agent makes no licence decisions and is never blocked by one. The binding constraint on the agent is therefore scientific honesty about what each asset actually is.

2. NON-NEGOTIABLE RULES

2.1 PRODUCT RULES

- P1. One object at a time. The renderer must be capable of showing exactly one

- OBJECT and must release the previous object's resources on switch.
- P2. Orbit, pinch-zoom, tap-to-focus, double-tap-to-reset must always work.
- P3. Two global controls besides the camera: OBJECT ROTATION (real sidereal period, speeds 1x / 60x / 3600x / hold) and SUN DIRECTION (azimuth, elevation, exposure in EV) with per-object presets.
- P4. Every dynamic visual effect must be a pure function of those two controls. No wall-clock animation anywhere in the app.
- P5. Time to first pixel after a button tap: under 400 ms. Time to full detail for a hero object: under 6 s on a flagship. No frame longer than 33 ms while tiles stream.
- P6. Additive integration. The feature adds new modules and two UI integration points; it changes nothing else in ZIG (§26.2, §26.3).
- P7. Independent by construction. No museum module reads, writes, imports or otherwise depends on any existing ZIG feature module, data store or calculation, and no existing module depends on a museum module except :app.

2.2 TECHNICAL RULES

- T1. Kotlin + Jetpack Compose. Rendering via Google Filament. No other engine.
- T2. All materials are authored offline with Filament's material compiler (matc) into .filamat files and shipped precompiled. No runtime shader compilation, no GLSL strings inside Kotlin.
- T3. All textures ship as KTX2 (ASTC for colour maps, UASTC where the encoder performs better for normal maps). Never a runtime PNG/JPEG decode for object surfaces.
- T4. Do not assume a texture dimension above 8192 in one axis. Wider data must be expressed as tiles, a sampler2DArray slice set, or the tile store.
- T5. Never block the main thread. Loading, decoding and I/O happen on Dispatchers.IO or a dedicated worker pool.
- T6. Every GPU resource acquired is released on object switch, on pause, and on destroy. Write the teardown path in the same commit as the setup path.
- T7. The release build must not request the INTERNET permission.
- T8. Units: each object is normalised to a scene radius of 1.0 (see §5.3). Camera distances are expressed in object radii. The ISS additionally exposes a metre scale bar using its true dimensions.

2.3 ASSET AND PROVENANCE RULES

- A1. LICENSING IS OUT OF SCOPE. The agent makes no licence decisions, maintains no licence registry, verifies no permissions and raises no licence-based blocker. The owner handles licensing after the feature is delivered, with legal counsel. Choose assets on quality alone (§27).
- A2. RECORD PROVENANCE. Every asset that ships needs, in its pack manifest: product name, publisher, credit string and the URL it came from. This is not a licence record - it is traceability, so the owner can identify, trace and clear every asset later. A pack entry missing any of those fields is a defect and fails the gate (§16).
- A3. Never invent an asset, dataset, product name or URL. If you cannot find the product page a dataset came from, say so (§23.4 S1) rather than guessing.
- A4. WRITE THE CODE YOURSELF. Third-party source code is never pasted, vendored or transcribed into this repository; external code enters only as a declared dependency at a pinned version (§3). Re-implement algorithms from published descriptions, cite the paper in a comment, and keep a note in docs/SOURCES.md. This applies to shaders in particular.
- A5. The provenance manifest drives the in-app Sources & Credits surface (§5.9): one row per shipped asset, generated from the manifests, never hand-written.
- A6. Never copy licence text, invoices, agreements or permission documents into

the repository or the app. Paperwork lives outside this repository.

3. PINNED TOOLCHAIN AND VERSIONS

Use exactly these versions unless a build incompatibility is proven; if you must change one, record the reason in docs/decisions/DECISIONS.md and report it.

- REVISION 2.0 NOTE - THIS TABLE WAS WRITTEN FOR A GREENFIELD PROJECT.
- ZIG already exists, so:
- FIRST establish what the repository actually uses (AGP, Kotlin, Compose BOM, JDK, navigation approach) in M-1 and record it in docs/integration/ENVIRONMENT.md, with file paths.
 - Do NOT upgrade, downgrade or realign the existing app's dependencies. If the new modules can use the existing versions, use them.
 - Where a new module genuinely needs something the app does not have (Filament, for instance), add it for that module only, at the version pinned below or the nearest version compatible with the app's existing toolchain, and record the decision.
 - Removing or bumping an existing dependency to satisfy this feature is forbidden (\$26.2).

Component	Version	Notes
Android Gradle Plugin	9.4.0	newest stable verified 2026-09-14; do not use alpha/beta/rc
Kotlin	2.2.0	
Compose BOM	2026.09.00	
compileSdk / targetSdk	newest stable (36)	no preview SDKs
minSdk	26	
Google Filament	com.google.android.filament:* 1.71.5	engine + gltfio + utils
SceneView	io.github.sceneview:sceneview 2.3.0	
Play Asset Delivery	com.google.android.play:asset-delivery 2.3.0	optional Compose/loading layer
KTX-Software	latest stable release at build time	
meshoptimizer	latest stable release	offline CLI tool (asset kit)
GDAL	3.8+	offline CLI/library (asset kit)
Python	3.11+	offline (asset kit)
Java toolchain	17	asset kit scripts

Dependency policy: add no third-party runtime dependency beyond what the feature genuinely needs, and record every addition in docs/decisions/DECISIONS.md with the reason. Prefer the AndroidX/Jetpack stack already implied above. No DI framework, no HTTP client, no serialization library beyond kotlinx.serialization if needed.

4. REPOSITORY ARCHITECTURE

4.1 WHAT TO ADD, AND WHERE

This section was written for a greenfield project. The repository exists, so read the tree below as the SHAPE of the new content rather than as literal paths: match the real repository's module naming, package roots and directory conventions, and record the mapping in docs/integration/TOUCH_POINTS.md before creating anything (section 26.4). EVERYTHING LISTED HERE IS NEW CONTENT. The only existing files that may be edited at all are the integration points of section 26.3.

```
<the app's root>/
  settings.gradle.kts
  build.gradle.kts
  gradle/libs.versions.toml
  app/
  core/

<- single source of versions
<- :app (EXISTING: two integration edits only)
```

```

engine/                <- :core:engine (Filament wrapper, no UI)
data/                  <- :core:data (manifests, packs, tiles)
model/                 <- :core:model (pure Kotlin domain types)
designsystem/          <- :core:designsystem (theme, components)
credits/               <- :core:credits (manifest -> UI surface)
feature/
  museum/              <- :feature:museum (grid + card destination)
  viewer/              <- :feature:viewer (the render surface)
  settings/            <- :feature:settings (quality, exposure)
tools/
  assetkit/            <- :tools:assetkit (JVM CLI, offline)
  blackhole-lut/       <- :tools:blackhole-lut (JVM CLI, offline)
  ci/                  <- :tools:ci (provenance + scope gates)
assets-src/            <- DOWNLOADED SOURCE DATA (git-ignored)
assets-built/          <- BUILT PACKS (git-ignored, checksummed)
manifests/             <- provenance manifests (VERSIONED, in git)
docs/
  decisions/DECISIONS.md <- one entry per architectural choice
  pipeline.md          <- how to rebuild every pack
tests/
  golden/              <- reference images + tolerance config
.github/workflows/     <- CI: build, lint, unit tests, gates

```

4.2 MODULE RULES

- :core:model has no Android dependency. Pure Kotlin data classes only.
- :core:engine owns every Filament type. Nothing outside this module imports `com.google.android.filament.*`.
- :core:data owns manifests, pack discovery and the tile store's storage side.
- :feature:* contains UI and state; it talks to :core:engine through interfaces defined in :core:engine (no direct engine calls from Composables).
- :tools:* are JVM-only, never shipped in the APK, and must be runnable with `./gradlew :tools:assetkit:run --args=...``.
- NO existing module gains a dependency on a museum module except :app, and no museum module may depend on an existing feature module (§26.2, P7).
- The rule that :core:engine owns every Filament type still applies: no other module in the app may import `com.google.android.filament.*`.

4.3 PACKAGE NAMING

Application id: `com.zig.inspector`
 Kotlin packages mirror module paths, e.g. `com.zig.core.engine`,
`com.zig.feature.viewer`, `com.zig.tools.assetkit`.

4.4 THE OBJECT REGISTRY (the spine of the app)

A single Kotlin object-registry file defines all thirteen objects as data:

```

ObjectSpec(
    id, displayName, sceneRadiusMetres, oblateness, axialTiltDeg,
    rotationPeriodHours (signed for retrograde), packId, qualityTiers,
    dataCeilingText, credits[] )

```

This file is the single source of truth for the UI, the loader and the credits screen. Adding an object must never require touching the renderer.

Values are taken from the IAU Working Group on Cartographic Coordinates and Rotational Elements report and NASA planetary fact sheets; cite the source in a comment next to each numeric literal. (Research doc §24.1.)

4.5 HOW TO ADD WITHOUT DISTURBING ZIG

This brief does not name the files you may edit, because it cannot: it was not written by someone looking at your repository. Instead, one test governs every

edit, applied file by file and line by line:

Does Space Museum fail to work without this change?

If the answer is not an unqualified yes, do not make the change. Necessary is not the same as convenient: "cleaner", "more modern", "already broken", "helps later" and "the linter prefers it" are not necessities.

The practical consequences:

- Prefer NEW code to MODIFIED code, and additions to rewrites: a new function, a new parameter with a default value, a new entry in an existing list.
- To make the feature reachable, make the smallest edit that achieves it, in the style and idiom the file already uses.
- New dependencies are added for the new modules only. Nothing existing is bumped, removed or realigned.
- New strings, drawables and icons live in the new modules' own resources.
- Record every pre-existing file you touch in docs/integration/CHANGED_FILES.md, one line each, with the necessity. That file is reviewed at every milestone; an entry whose reason does not survive the test above is a defect to undo.
- If the feature cannot be added without a substantial rework of existing code - a redesign, a refactor, a rework of a flow - that is HARD STOP S8 (\$26.7).

=====

END OF PART I (sections 1-4). PART II follows: runtime architecture, materials, tile store, asset kit, per-object specifications.

=====

=====

PART II – TECHNICAL SPECIFICATION

=====

=====

5. RUNTIME ARCHITECTURE

=====

5.1 ENGINE OWNERSHIP AND LIFECYCLE

- One Filament Engine for the process, created in :core:engine behind an InspectorEngine class. Never create an Engine per screen.
- Engine creation happens off the main thread; the Compose surface attaches when ready. Show a deterministic placeholder until the first frame.
- The render loop is driven by a Choreographer-based frame callback or Filament's own swap-chain loop via UiHelper/SurfaceView. Pick one and document it in docs/decisions/DECISIONS.md.
- Lifecycle: onViewDetachedFromWindow -> release View-side resources; on Activity destroy -> destroy Engine. Object switch -> destroy the object's renderables, textures and material instances, keep the Engine.

5.2 THE SCENE, PER OBJECT

A single Scene containing exactly:

- one object renderable (sphere/ellipsoid/torus/plane/model as appropriate),
- optional atmosphere shell mesh,
- optional ring or cloud meshes,
- a skybox (the deep-sky map, or the black-hole lensing shader),
- one directional light (the sun) and one optional fill light.

No other geometry. If something needs to be added, justify it in DECISIONS.md.

5.3 UNIT-RADIUS NORMALISATION (mandatory, this replaces floating origins)

- At load, the object's mean radius maps to 1.0 scene unit. All object-local

coordinates are in object radii.

- The camera works in object radii. Near plane: 0.0005 radii when inspecting a surface patch, up to 0.05 when framing the whole body. Far plane: 8.0 radii. Compute near/far from the current zoom each frame within these bounds.
- Double precision is used ONLY in offline tools and at load-time conversion. The frame loop is float. Record the conversion formula in DECISIONS.md.
- The ISS keeps true metre dimensions internally and exposes a metre scale bar; it is normalised the same way for camera purposes.

5.4 CAMERA RIG

- Orbit camera: spherical coordinates around the object centre with radius, yaw, pitch; damping; pinch to change radius logarithmically; two-finger twist optionally rolls the camera (off by default).
- Tap-to-focus: ray-cast from the tap, find the surface point on the object (analytic sphere/ellipsoid intersection, then refine against the current tile's height data if available), then smoothly recentre the orbit target to that point while preserving the view orientation. Double-tap resets to the framed default.
- Zoom limits: minimum radius = (2 x the object's current data resolution in radii) so the user can reach the CEILING but the app never lets the camera clip into geometry; maximum radius = 3.0 radii.
- Per-object camera presets: full disk, pole-on, terminator view, hero region (for objects that have one). Expose as buttons; each preset is data, not code.

5.5 THE RENDER PIPELINE (fixed order, do not reorder without a decision entry)

1. Clear to deep space black (never pure 0,0,0; use a very low but non-zero value so dithering and bloom thresholds behave).
 2. Skybox: deep-sky map (unlit cubemap/equirect material) OR the black-hole lensing material. Both are rendered as Filament skybox materials with depthWrite false.
 3. Opaque object geometry (sphere/ellipsoid/model) with its surface material.
 4. Opaque overlays: hero terrain patches, ring plane (if configured opaque), horizon-map shadow contributions are inside the surface material.
 5. Translucent layers in this order: cloud layer, atmosphere shell (back faces), ring plane (if translucent), atmosphere shell (front faces).
 6. Emissive additions: Earth night lights, Sun corona, aurora.
 7. Post: bloom -> tone mapping -> colour grading -> TAA -> dither -> UI overlay.
- Filament provides bloom, tone mapping (AgX, PBR Neutral, ACES, filmic), colour grading, TAA and SSAO through View options; verify exact API names and setters against the pinned 1.71.5 javadoc/KTX and record the names you used in DECISIONS.md. Do not guess method names; if a feature is not exposed, implement its equivalent as a post-process material and document that.

5.6 LIGHTING MODEL

- One directional light = the Sun. Intensity in physical Lux scaled by the object's real heliocentric distance: use relative irradiance factors (Earth = 1.0): Mercury 6.7, Venus 1.9, Earth 1.0, Moon 1.0, Mars 0.43, Jupiter 0.037, Saturn 0.011, Uranus 0.0027, Neptune 0.0011.
- The Sun object itself has no directional light dependence; it is emissive.
- Optional fill light per object, physically justified only: earthshine for the Moon (albedo 0.30, labelled approximation), and nothing else unless a decision entry justifies it.
- Exposure is user-controlled (EV slider) and every object stores a sensible default EV so it looks correctly exposed on open. Deep space stays black.
- No ambient floor. Shadowed regions are black except where a justified fill light or an atmosphere term exists.

5.7 MATERIAL LIBRARY (owned by :core:engine, authored offline)

One material per visual family, compiled to .filamat and shipped:

M1	regolithSurface	rocky bodies: albedo + normal + height + A0 + horizon-map shadows + optional displacement
M2	icySurface	high-albedo surfaces with subsurface scattering
M3	gasGiantSurface	banded flow, wind shear, methane absorption tint
M4	cloudDeck	opaque/translucent cloud layer with self-shadowing
M5	ringTransmission	optical-depth transmission, phase asymmetry, shadows
M6	solarSurface	procedural photosphere, limb darkening, spots
M7	coronaShell	optically thin emission with radial falloff
M8	nightLights	additive emissive layer (Earth)
M9	starSprite	point sprite with flux-preserving PSF
M10	atmosphereShell	LUT scattering, shell + surface terms
M11	blackHoleLens	precomputed-table beam tracing (skybox domain)
M12	modelSurface	glTF PBR for the ISS, with clearcoat/foil variants
M13	patchSurface	hero-region terrain patches with their own VT slot

Every material must declare its required attributes and samplers and must ship at least two variants: quality tier 0/1 and a reduced tier 2/3 variant where cheap enough. Compile with matc; keep the .mat sources in core/engine/src/main/materials/ and the build step in the Gradle task graph.

5.8 THE OBJECT LOADER (interface, then implementations)

```
interface ObjectSource {
    suspend fun load(spec: ObjectSpec, quality: QualityTier): ObjectHandles
    suspend fun release(handles: ObjectHandles)
}
```

Implementations:

SphereSource	generated ellipsoid + tile pyramid (most objects)
RingedSource	SphereSource + ring annulus + shadow parameters
ModelSource	glTF via gltfio (ISS) with LOD selection
SkySource	inside-out sphere / cubemap (Milky Way)
LensingSource	black-hole skybox material + disk geometry

The loader must be cancelable (structured concurrency). Switching objects before the previous load finishes must cancel cleanly and release partial resources. Write a unit test that proves it.

5.9 THE UI SURFACE (minimum viable, then polish)

- Space Museum screen: the destination opened by the Lab screen card. A Compose grid of thirteen tiles, each with the object name and its data ceiling as a subtitle (e.g. "Moon - 0.5 m/px at Apollo sites"), under a short museum header in the app's own design language. No network, no thumbnails fetched at runtime; a small local icon or a static bundled thumbnail per object is fine. The card and this screen are the only UI surfaces ZIG previously knew nothing about. The card itself is specified in section 26.5.
- Viewer screen overlays:
 - * top bar: object name, back, credits (i)
 - * bottom sheet or rail: rotation control, sun-direction control (azimuth, elevation, EV), toggles for the object's layers, camera presets
 - * DATA HUD: "DATA 166 m/px (MESSENGER MDIS BDR)" updating with zoom, and a "beyond published data - procedural detail" indicator that appears past the ceiling, with a toggle to switch procedural layers off (RULE: this indicator must be driven by the object's declared ceiling, not by a hard-coded zoom threshold).
 - * performance overlay (debug builds only): fps, frame time p50/p95, resident tile count, VRAM estimate.

- Sources & Credits surface: one row per shipped asset, generated from the installed packs' provenance manifests (\$16): object, product, publisher, source URL, credit string, what the data means, and the processing applied. Never hand-written. This is a product feature - it tells the user where the data came from - and it is also what lets the owner trace every asset later.

6. THE OFFLINE ASSET KIT (:tools:assetkit)

6.1 PURPOSE AND RULE

The asset kit turns primary scientific data into shippable packs. It runs on a workstation, never on the device, and it must be deterministic and reproducible: same inputs + same config => byte-identical outputs (verify by hashing twice).

6.2 CLI SURFACE (implement exactly these verbs)

```
assetkit fetch --object moon --source wac-100m      (downloads + verifies)
assetkit preprocess --object moon --config configs/moon.yaml
assetkit tiles --input <preprocessed.tif> --out <pack-dir> --tile 512 --apron 4
assetkit encode --input <dir> --format astc --block 6x6
assetkit horizon --height <dem.tif> --azimuths 16 --out horizon.ktx2
assetkit normal --height <dem.tif> --out normal.ktx2 --strength <float>
assetkit pack --object moon --out assets-built/moon.zigpack --manifest ...
assetkit verify --pack assets-built/moon.zigpack      (checksums + manifest)
```

Every verb must support --dry-run and must print exactly what it would do.

6.3 PACK FORMAT (.zigpack)

A .zigpack is a ZIP (stored, no recompression of already-compressed KTX2) containing:

manifest.json	provenance + geometry metadata (Appendix A)
maps/<name>/*.ktx2	full-resolution maps (albedo, normal, height, masks)
tiles/L<level>/<x>_<y>.ktx2	tile pyramid where used
luts/*.ktx2	atmosphere or lensing LUTs
meta/tiers.json	per-quality-tier map selection

Rule: the pack must be usable with a plain ZipFile read on device. No custom container formats, no per-file compression of KTX2.

6.4 TILE PYRAMID RULES

- Tile size 512x512 texels, apron 4 texels on every side (bleed into neighbours to prevent seams under filtering). Store the apron; do not trim.
- Levels: L0 = native resolution of the source, each subsequent level halves the resolution until the width is <= 512. Name tiles by integer x,y in the level grid, y increasing upward in the equirectangular convention.
- Naming: tiles/L<level>/<x>_<y>.ktx2 with zero-padded indices is acceptable but must be consistent and documented. The reader must not hard-code the padding width.
- The equirectangular seam at longitude 0/360 must be handled by duplicating the first column of tiles at the end of each row (an explicit duplicate, not a wrap assumption).
- Poles: do not let the polar rows collapse. Either oversample the polar caps or document the chosen pole treatment in the manifest.

6.5 ENCODING RECIPES (verify flags against the pinned KTX-Software release)

Albedo (sRGB, perceptual):

```
ktx create --format R8G8B8_SRGB --encode astc --astc-blocksize 6x6 \
  --generate-mipmap --assign-tf srgb --zstd 18 in.png -o out.ktx2
```

```

Albedo hero (highest quality):
  same with --astc-blocksize 4x4
Normal maps (two-channel, best fidelity):
  ktx create --format R8G8_UNORM --encode uastc --uastc-quality 4 ...
  (or ASTC 6x6 after visual verification; document which you chose and why)
Height / masks (single channel):
  ktx create --format R8_UNORM --encode astc --astc-blocksize 6x6 ...
HDR sky (no ASTC HDR on most devices):
  ktx create --format R11F_G11F_B10F ... (or RGB9_E5)
LUTs:
  ktx create --format R32G32_SFLOAT ... (lensing tables), R16_SFLOAT (atmos)
Rules: always generate mips in linear space; always state the transfer
function; never ship an sRGB-encoded normal map or mask.

```

6.6 HORIZON MAP ALGORITHM (implement as `assetkit horizon`)

Input: a height field (metres) on a regular lat/lon grid.
 For each texel and each of N azimuth directions (N = 16 recommended):
 walk outward along the azimuth up to a maximum angular distance (suggested 128 texels at the native DEM resolution), compute the maximum elevation angle $\text{atan}((h_i - h_0) / \text{distance_in_metres})$, and store the maximum.
 Output: an RGBA or N-channel R8 texture containing the elevation angle quantised to 0..255 over 0..90 degrees. Provide both a 16-channel form (as a 2x RGBA8 texture set or one 8-channel texture used twice with an offset) and a simpler 8-channel form.
 Runtime use: in M1 regolithSurface, if the sun elevation is below the horizon angle for the sun's azimuth bin, the texel is in shadow. Interpolate between the two nearest azimuth bins. The same map drives ambient occlusion as (mean horizon angle) -> visibility, which darkens crater floors.
 Determinism: the walk must be in metres, computed from the raster's own georeferencing, never in texel units with an assumed aspect ratio.

7. THE TILE STORE (runtime, :core:data + :core:engine)

7.1 DATA STRUCTURES

```

TileKey(objectId, level, x, y)
TileEntry(key, textureHandle, lastUsedFrame, byteSize, state)
  state in { REQUESTED, DECODING, RESIDENT, EVICTABLE }
TileStore(objectId):
  - resident map TileKey -> TileEntry
  - request queue ordered by priority
  - LRU list for eviction
  - hard budget in bytes (from the quality tier, $15)

```

7.2 LEVEL SELECTION

Per visible screen region, compute projected texel density:

$$\text{texelsPerPixel} = \frac{(\text{tileTexelsPerRadian} \cos(\text{latitude}) \ 2\pi * \text{radius_px})}{(2 \ \pi \ \text{objectRadiusPx} * \dots)}$$

 In practice: implement a function that, for a given camera and object, returns the desired level L for each tile in the visible set, then clamp L to the pack's available levels. Provide unit tests with hand-computed expectations for three cameras: full disk, half zoom, surface inspection.

7.3 SCHEDULING, PREFETCH, NO-HITCH RULES

- Decode on a fixed-size worker pool (suggested 3 threads) using

- Dispatchers.IO; never on the render thread.
- Upload to the GPU on the render thread only, at most K textures per frame (K = 2 for tier 0, 1 for tiers 1-3), so no frame exceeds the frame budget.
- Prefetch: when the camera moves, enqueue the next level up for tiles near the screen centre first, then along the motion vector.
- Eviction: LRU with a protected set for the tiles currently visible at the desired level. Evict before allocating if the budget would be exceeded.
- Stall policy: if the desired level is not resident, render the resident coarser level (mip-style fallback) rather than waiting. NEVER block a frame.
- Instrumentation counters: residentTiles, residentBytes, queuedTiles, decoderQueueDepth, evictionsPerFrame, uploadsPerFrame, fallbackFrames.

7.4 VIRTUAL TEXTURE ALTERNATIVE

Filament supports virtual texturing (streaming textures produced by its tooling). Evaluate it in milestone M3 against the hand-written tile store and pick one based on measured numbers (quality, code size, memory behaviour). Whichever is chosen, the acceptance criteria in M3 apply unchanged.

8. MATERIAL SPECIFICATIONS (authoring notes for matc)

All materials are written in Filament's material language (the .mat file plus documentation available in the pinned engine release). Consult the engine's Materials reference for the exact parameter syntax, the available shading models (lit, unlit, subsurface, cloth), parameter types (sampler2d, sampler2dArray, samplerCubemap), the customSurfaceShading mechanism, and the vertex block (`materialVertex`) used for displacement. A verified fact from the pinned engine documentation: custom vertex blocks CAN modify geometry (for example worldPosition), and the material flag that skips custom vertex work on depth-only passes must NOT be enabled for materials whose vertex block moves geometry, or shadows will be wrong. Record this in DECISIONS.md when you use it.

8.1 M1 regolithSurface (Moon, Mercury, Mars, Venus radar mode)

Inputs: albedo (sRGB), normal (tangent space, OpenGL convention +Y up), height (R16F or R8), AO (R8), horizon (N x R8), roughness scalar, albedo scale scalar, proceduralStrength scalar, proceduralScale scalar.

Requirements:

- Displacement: vertex block samples the height map with a level-aware LOD bias, displaces along the vertex normal by (height - referenceRadius) * displacementScale. Reference radius comes from the pack manifest so that displacement is in real metres scaled to object radii.
- Must accept an optional UV offset/scale so the same material serves hero patch geometry that covers a sub-region.
- Normals: recompute the tangent frame perturbation from the height derivative at the displaced resolution, or use the shipped normal map (choose by tier).
- Shadow: horizon-map lookup as described in §6.6; multiply direct sun by 0 or 1 with a smooth step of about 1 degree to avoid aliasing.
- AO multiplies ambient/indirect only, never direct sun.
- Procedural micro-detail: optional, off by default for "science" modes, on for "inspection" mode, always reported through the data HUD state.
- Micro-detail must use a noise field that does not tile visibly at any zoom (domain-warped, scale-locked to object radii, never to screen space).

8.2 M3 gasGiantSurface (Jupiter, Saturn, Uranus, Neptune)

Inputs: albedo map, wind LUT (latitude -> angular velocity), flowScale, shearAmount, methaneTint, limbParams, oblateness is geometry not material.

Requirements:

- Sample the albedo with a latitude-dependent longitude offset driven by the wind LUT to produce real band shear; add curl-noise advection for sub-map detail below the data ceiling only.
- Limb treatment: darken toward the limb with a methane-absorption term and a

bright haze band; use the atmosphere material for the outer glow.

- Anisotropy is allowed (Filament supports it) for a banded sheen.
- No displacement, no normal-map relief: the cloud tops are flat relative to their scale; state this in comments so nobody "improves" it later.

8.3 M5 ringTransmission (Saturn, Uranus, Neptune)

Inputs: opticalDepthProfile (2D or 1D + azimuth dimension), opticalDepthScale, colorMap, forwardScatterBias, thicknessMeters, planetShadow params, ringShadowLookup, azimuthalVariationStrength.

Requirements:

- $\alpha = 1 - \exp(-\tau / \mu)$ with $\mu = \cos$ of the incidence angle computed per fragment from the true sun direction.
- Two-sided rendering with a phase-dependent brightness asymmetry (forward-scattered light is brighter). Document the chosen phase function.
- Planet shadow on ring: analytic test of the sun ray against the planet ellipsoid; where inside, multiply by the shadowed factor.
- Ring shadow on planet: implemented in M1/M3 via a lookup that takes the fragment position, projects it along the sun direction onto the ring plane, converts to a ring radius, and samples tau; the shadow factor multiplies the direct sun term.
- Thickness: geometry is a plane for the main rings. A volumetric look is permitted only for the E ring and only if the budget allows.
- Spokes: optional azimuthal modulation, off by default, labelled when on.

8.4 M6 solarSurface and M7 coronaShell

- Photosphere: 3-octave domain-warped noise for granulation, animated in a third time-like dimension (never by scrolling UVs); supergranular network mask; sunspots injected from a small table of real active regions with umbra and penumbra temperatures (about 4000 K and 5000-5500 K).
- Limb darkening: use published limb-darkening coefficients (a 1D polynomial in μ) rather than an artistic curve.
- Emission: output radiance well above the scene's mid grey so tone mapping and bloom engage; expose an "emission scale" parameter so the exposure control still works.
- Corona: optically thin shell with an analytic radial falloff plus structure driven by a low-resolution synoptic magnetogram field; keep the falloff exponent a parameter with a default justified in comments.
- The Sun must never be a flat yellow sphere with a texture.

8.5 M10 atmosphereShell + the scattering LUTs

- Implement single + multiple scattering using LUTs (a 2D transmittance LUT and a 3D multi-scattering LUT). Generate the LUTs offline in the asset kit (tool: `assetkit atmosphere`) and ship them in the pack. Reference method: the Bruneton/Hillaire approach. Implement it yourself from the published equations and cite the paper in a comment (A4): no third-party shader source is vendored into this repository.
- Per-body parameter sets ship in the manifest: Rayleigh coefficients and scale height, Mie coefficients, asymmetry g , absorption species and their extinction, ground albedo for the bounce term.
- Two usages: a surface term (fogs the terrain toward the limb and softens the terminator) and a shell mesh (the limb glow against space). Both must use the same parameters, and the shell must render back faces before the object and front faces after it.
- Earth must include ozone absorption; the sunset band is the acceptance test. Venus must be Mie-dominated and must hide the surface. Mars must be dust-dominated with a variable opacity parameter. Gas giants need only limb

treatment plus methane tint.

8.6 M11 blackHoleLens – see §12 for the full specification.

9. DISPLACEMENT, RELIEF AND LEVEL OF DETAIL

9.1 THE APPROACH (settled, do not re-litigate without evidence)

Primary: runtime GPU displacement in the vertex block of M1, sampling the height map with a LOD bias appropriate to the current level. This is what makes inspection possible without shipping dozens of pre-baked meshes.

Fallback: if vertex texture fetch performance fails the M11 acceptance criteria on target devices, bake a 4-level displaced mesh chain offline and select by camera distance. Record the measurement that forced the fallback.

9.2 MESH GENERATION RULES

- Base geometry: a UV sphere or ellipsoid generated in code (not a glTF asset), segment counts per tier: 256x128 (tier 0), 192x96 (tier 1), 128x64 (tier 2), 96x48 (tier 3). Equirectangular UVs must match the texture convention exactly; unit-test the UV of a known lat/lon.
- Oblateness: apply the object's flattening ratio to the vertical axis before any displacement. Verify against the registry values.
- Poles: use a pole-aware triangulation (or accept the standard UV sphere and document the pole artifact) so the texture seam and the pole pinch are not visible at close zoom. If the distortion is unacceptable at the ceiling zoom for an object, that object gets a cube-sphere mesh instead, recorded in DECISIONS.md.

9.3 HERO PATCHES

For objects with metre-scale regional data (Moon NAC, Mars HiRISE, Earth Landsat), geometry for a hero region can be a separate patch that is a high-resolution displaced grid in the object's local frame, using M13 patchSurface. Patches must:

- share the object's lighting and camera code,
- carry their own manifest entry with the same provenance fields (A2),
- be positioned by lat/lon converted through the same code path as the sphere UVs (unit-test that a patch at a given lat/lon lands where the texture says).

END OF PART II SECTIONS 5-9. Continues: light geometry, atmospheres, black hole, stars, per-object specs.

10. LIGHT GEOMETRY: SHADOWS, TERMINATORS, RING SHADOWS

10.1 PRINCIPLE

Every body here is a sphere/ellipsoid and every ring is a plane, so shadows are computed ANALYTICALLY in the shader, not with shadow maps. This is sharper, has no cascade artifacts at any zoom, and costs a few ALU operations. Reserve Filament shadow maps for the ISS and for local hero patches only.

10.2 THE FIVE EFFECTS, WITH ACCEPTANCE TESTS

- E1 Terminator. Airless bodies: hard N.L terminator with correct falloff.
Acceptance: the terminator is a clean arc with no banding at any zoom.
- E2 Terrain self-shadowing (horizon map). Acceptance: at a sun elevation of 5 degrees over a crater field, crater interiors are black and rims catch

light; the shadow does not crawl or shimmer when the camera moves (this is the test that catches an incorrectly computed azimuth bin).

- E3 Ring shadow on planet. Acceptance: for a chosen sun geometry, the shadow band's position and width match a hand-computed analytic expectation within one tile.
- E4 Planet shadow on rings. Acceptance: the shadow is an ellipse, not a circle, and its edge is smooth under zoom.
- E5 Limb darkening (Sun) and limb haze (gas giants). Acceptance: the Sun's disk centre-to-limb falloff matches the published coefficient curve within 2%.

10.3 SUN-DIRECTION CONTROL (the physics, so it is implemented once correctly)

- State: azimuthDeg, elevationDeg, exposureEV.
- The world sun direction is derived from these two angles in the object's local frame; because there is no orbit, the frame is defined as "object-fixed, north up, prime meridian toward the viewer's default camera".
- Implement as a single pure function:
`sunDirection(azimuthDeg, elevationDeg): Vec3`
 with unit tests covering the cardinal directions and the poles.
- Presets per object store (azimuth, elevation, EV) triples, not images.

11. ATMOSPHERIC SCATTERING: OFFLINE LUT PIPELINE

11.1 OFFLINE STEP (`assetkit atmosphere`)

Input: a parameter file per body (Appendix C lists the parameters to expose).

Output: transmittance LUT (2D, R16_SFLOAT, suggested 256x64),
 multi-scattering LUT (3D, R16_SFLOAT, suggested 32x32x32),
 and an optional sky-view LUT per body if the chosen technique needs it.

The generator must be deterministic and must print the physical inputs it used. It must be a clean-room implementation from the published technique's equations (A4: written here, from the published equations, citing the paper). Do not vendor third-party shader source.

11.2 RUNTIME STEP

Two shader stages, one parameter set: surface term inside M1/M3, shell term in M10. The shell renders back faces before the object and front faces after it. Acceptance: Earth's limb is blue; the terminator shows a warm band; a low sun produces a reddening that transitions through orange rather than jumping to red; Venus's surface is hidden at all angles; Mars's dust produces a butterscotch sky that responds to the dust-opacity parameter.

11.3 TIERS

- Tier 0: full LUT multiple scattering at full resolution.
- Tier 1: same LUTs at half resolution, upsampled.
- Tier 2: single scattering only.
- Tier 3: analytic Rayleigh approximation (visibly simpler, still labelled).

12. BLACK HOLE SUBSYSTEM (highest risk; build it as a module)

12.1 PHYSICS TO IMPLEMENT, AND WHAT EACH PRODUCES

- Critical impact parameter $b_c = 3\sqrt{3}M$ (Schwarzschild), i.e. a shadow radius of $2.598*r_s$. This sets the shadow size; assert it in a unit test.
- Deflection of light: precomputed tables mapping impact parameter and radial

- coordinate, so the fragment shader can find the disk-plane intersection and the exit direction in constant time per pixel. (Bruneton 2020 method: implement it yourself from the paper, per A4. Do not copy the reference implementation's source into this repository.)
- Photon ring: emerges from the tables; do not fake it with a ring mesh.
- Accretion disk: thin annulus from the ISCO outward; temperature profile $T(r)$ proportional to $r^{-3/4}$; emitted spectrum is a blackbody evaluated through a precomputed blackbody-to-RGB table (offline, in the asset kit).
- Doppler + gravitational shift: compute the total shift factor g from the orbital velocity at the emitting radius and the gravitational potential, then transform BOTH the temperature and the intensity. Use the Liouville invariant consistently: either I proportional to g^3 (per-frequency convention) or g^5 (per-wavelength convention). Pick one, state it in comments, and use it everywhere. Acceptance: the approaching side is brighter and bluer, the receding side dimmer and redder, and the asymmetry magnitude changes with inclination exactly as expected.
- Background lensing: sample the deep-sky map with the deflected direction; stars must compress into the Einstein-ring pattern with a secondary image inside the ring.
- Higher-order images: appear automatically; verify at least two of them.

12.2 OFFLINE LUT GENERATOR (:tools:blackhole-lut)

Produce: $D(e,u)$ table (suggested 512x512, RG32F) and $U(e,\phi)$ table (suggested 64x32, RG32F), plus the blackbody colour table (suggested 256 entries, RGBA16F). Requirements: deterministic; a --verify verb that integrates a set of test rays numerically and compares against the table to a stated tolerance; the tolerance and the comparison result recorded in the output metadata.

12.3 SHADER

Domain: skybox material (renders behind everything, no depth write) with the camera position supplied as a variable. The disk is drawn as geometry with its own material so it can be occluded correctly and so bloom treats it as a real HDR source.

Tiers:

- Tier 0: full tables, full-resolution disk sampling, multiple scattering ignored, photon-ring over-brightening pass enabled.
- Tier 1: half-resolution lensing pass upsampled, reduced disk samples.
- Tier 2: quarter-resolution lensing, analytic weak-field deflection outside a radius threshold, no over-brightening pass.
- Tier 3: baked deflection field for the current camera radius, re-baked when the radius changes by more than 20%.

User control: a "ray steps / table quality" slider with named presets ("Cinematic (physically correct)", "Balanced", "Battery saver") and an fps readout.

12.4 MODES AND LABELLING (non-negotiable)

- "Physically correct": beaming and redshift on, parameters honest.
- "Cinematic": reduced Doppler asymmetry, documented after James et al. 2015 (their visualisation used $a/M = 0.6$ and a 6500 K white balance, and the paper documents suppressing the Doppler intensity change). The UI must state that this mode is a documented artistic variant, not a physical prediction.
- "M87-like" and "Sgr A-like": low-inclination photon-ring-dominated views with radio-wavelength emission and GRMHD-informed profiles, citing the EHT measurement of the M87* shadow (42 +/- 3 microarcseconds) on the Sources & Credits surface.
- Never describe any mode as "the Interstellar black hole" and never ship any

asset from the film.

12.5 VALIDATION (acceptance, do not skip)

- Unit test: b_c and the shadow radius.
 - Table verification verb passes within tolerance.
 - Comparison against an independent CPU reference renderer for at least three configurations (face-on, 60 degrees, near-edge-on): the disk's far-side arc over the shadow, the photon ring radius, and the brightness ratio between the approaching and receding sides must match within a documented tolerance.
- An existing open-source Kerr/Schwarzschild renderer may be used as the comparison target, run locally as a tool; per A4 it is never linked into the shipped app and never copied into this repository.

13. STAR SYSTEM (Milky Way object, and the black hole's sky)

13.1 DATA

- Deep map: NASA SVS "Deep Star Maps 2020" (item 4851). Use only layers whose provenance is not Gaia-derived; if the chosen file bundles Gaia content, choose a different layer or a different source. Record the exact file used.
- Catalogue stars: USNO UCAC4 and Yale Bright Star Catalogue 5th edition (public domain / US Government works). Extract: unit direction, magnitude, colour index, proper motion where available, designation.
- Constellations: IAU 1930 boundary data and constellation figures from the SVS item; star names from the IAU Working Group on Star Names list.
- SELECTION (revision 3.0): use the best star data available, whatever its provenance (§27.4). Gaia DR3 and the Digitized Sky Survey raise the achievable star count and the accuracy of the bright-star layer; the SVS deep map remains the fallback. Licensing is not your concern (A1); record the source per A2.

13.2 RUNTIME

- Deep map as an ASTC LDR cubemap or equirect skybox; carry HDR range in the star sprites, not the sky.
- Stars: one interleaved vertex buffer (direction as 3 x float16 or packed normal, magnitude, colour index), drawn as one instanced draw. Fragment shader implements a flux-preserving point-spread profile: a sprite must keep constant total flux as its screen size changes, and must never disappear when sub-pixel (clamp to a minimum size with flux compensation).
- Zoom behaviour: magnification resolves member stars of clusters; it must not magnify the deep map into blur. Acceptance test: zoom into the Pleiades and confirm resolved bright stars with correct colours against a reference list.
- Constellation overlay: line primitives with an opacity control; boundaries as a separate toggle; labels rendered with a screen-space text pass and occlusion-free sorting.
- Star picking: tap a star, show designation, magnitude, colour index and distance where known. Selecting must be stable under zoom.

14. PER-OBJECT SPECIFICATIONS

Each object section states: DATA (what to fetch), GEOMETRY, MATERIALS, CONTROLS, PACK CONTENTS, and ACCEPTANCE. Fetch rules: use the product's own archive or distribution page, record the provenance (product, publisher, URL), never substitute silently.

REVISION 3.0 NOTE. The DATA lines below were written when only public-domain and CC-free material could be shipped, and they carry that history in their wording. IGNORE EVERY LICENCE-BASED RESTRICTION IN THEM. Licensing is not part of this build (§2.3 A1): where section 27.4 names a better asset, use it; where a DATA line says "do not use X", "derive from archived data instead" or "prefer the lower-resolution route because its terms are simpler", treat that as obsolete and choose on quality. What still binds is scientific honesty: never present one kind of data as another (true colour, enhanced, radar, thermal, simulated, procedural), and never claim resolution the data does not have.

14.1 SUN

DATA: SDO/AIA and HMI frames (JSOC; CC0 waiver; requested credit "Courtesy of NASA/SDO and the AIA and HMI science teams"). 4096x4096 frames. Optional Carrington EUV map (NASA SVS item 30362), used only in a labelled EUV mode. GEOMETRY: sphere, 64x32 base (the detail is in the shader), no displacement. MATERIALS: M6 solarSurface, M7 coronaShell, star sprite sky. CONTROLS: rotation (Carrington period); channel selector (photosphere, AIA 171 / 193 / 304 labelled as false colour, HMI magnetogram); corona intensity; emission scale. PACK: base 2K procedural + one AIA disk (about 8 MB); install-time 8K procedural + 10 frames + magnetogram (about 120 MB); on-demand 100-frame series. ACCEPTANCE: granulation is visible and stable (no boiling); limb darkening matches coefficients within 2%; the disk clips to white under default exposure with bloom; no yellow flat texture anywhere; EUV modes are labelled.

14.2 MERCURY

DATA: USGS Astrogeology MESSENGER MDIS Basemap BDR 166 m (92160x46080, 4 GB) and LOI 166 m (4 GB); Global DEM 665 m (506 MB); MD3 colour 665 m (760 MB); enhanced colour 665 m (759 MB, labelled false colour). All public domain, "cite authors". GEOMETRY: ellipsoid, radius 2439.4 km, near-spherical; displacement from the DEM. MATERIALS: M1 regolithSurface (M13 for hero patches if any). CONTROLS: BDR/LOI switch, MD3/enhanced switch (the latter labelled), DEM and normal overlay, sun angle, horizon shadows toggle. PACK: base 8K BDR downsample + 4K MD3 (about 25 MB); install-time 16K BDR + 8K DEM + 8K normal (about 260 MB); on-demand full 166 m/px pyramid. ACCEPTANCE: the data HUD reads 166 m/px when the pyramid is installed and about 937 m/px otherwise; relief detail stops at 665 m/px and the HUD states it; enhanced colour is labelled.

14.3 VENUS

DATA: Magellan C3-MDIR 4641 m colourised radar (96 MB); Global Topography 4641 m, 16-bit (65 MB); Fresnel reflectivity (33 MB); meter-scale slope (33 MB); optional stereo-look 75 m mosaic (82 GB, about 17% coverage). GEOMETRY: sphere, radius 6051.8 km; displacement from GTDR. MATERIALS: M1 (radar mode), M4 cloudDeck, M10 atmosphere (Mie-dominated). CONTROLS: cloud deck (default, labelled) vs radar surface (labelled) vs topography; Fresnel and slope overlays; atmosphere on/off; peel transition. PACK: base 4K radar + cloud shader (about 10 MB); install-time 8K radar + 8K topography + masks (about 180 MB); on-demand 17% stereo patches. ACCEPTANCE: the default view is a nearly featureless pale cloud deck; radar mode is labelled as radar; the peel transition explains the two; no visible-light surface imagery is implied anywhere.

14.4 EARTH

DATA: Blue Marble Next Generation 21600x10800, 12 monthly composites; Black Marble 2016 night lights (up to 86400x43200, use a sensible tier); ETOPO 2022 for bathymetry; SRTM or ETOPO for land relief; LANDSAT Collection 2 for hero tiles at 15-30 m/px over land (USGS; credit "Data available from the U.S. Geological Survey"). Sentinel-2 and other ESA-processed imagery may be used

for hero tiles where they improve the result (A2).

GEOMETRY: ellipsoid, WGS84-ish flattening; displacement from land relief plus bathymetry (subtle, mostly for the limb).

MATERIALS: M1 with an ocean mask driving roughness (ocean roughness 0.02-0.1, IOR 1.33, Fresnel sun glint), M8 nightLights, M4 cloudDeck, M10 atmosphere with ozone.

CONTROLS: month selector (12 labelled BMNG months, not "today"); day/night crossfade; cloud opacity; ocean glint; bathymetry overlay; atmosphere; hero tiles; sun angle.

PACK: base 8K BMNG + 4K night + 4K clouds (about 45 MB); install-time 12 months at 8K + 8K night + 8K normal + clouds (about 420 MB); on-demand Landsat hero tiles (64-128 tiles, about 0.6-1.4 GB).

ACCEPTANCE: ocean has a real specular glint that moves with the sun control; night lights appear only on the dark side and are dimmed under cloud; the month selector visibly changes the snowline; Landsat tiles reach 15-30 m/px with the HUD showing the correct figure; bathymetry is labelled as modelled.

14.5 MOON

DATA: LROC WAC global morphology mosaic 100 m/px (109152 x 54576; via USGS Astrogeology, the PDS LROC node, or the LROC curated downloads - all three routes are in play); LOLA ldem_64 and the 59 m selenoid height field; NASA SVS "CGI Moon Kit" colour map (16K) and displacement map; LROC NAC imagery and NAC DTMs at 0.5 m/px for hero sites. The NACs at 0.5 m/px are the finest planetary surface data in existence: they are the showcase of this object.

GEOMETRY: sphere, radius 1737.4 km; displacement from LOLA; hero patches for Apollo sites, Tycho, Aristarchus, Reiner Gamma, Hadley Rille.

MATERIALS: M1 regolithSurface with horizon-map shadows (this is the showcase).

CONTROLS: WAC morphology vs SVS colour vs Hapke 7-band (labelled); LOLA overlay; hero-patch navigation (jump to site); sun angle with a grazing preset; horizon shadows toggle.

PACK: base 8K WAC + 4K normal (about 40 MB); install-time 16K WAC + 8K LOLA + 8K normal + horizon map (about 520 MB); on-demand 100 m/px global pyramid plus 20 NAC patches at 0.5 m/px.

ACCEPTANCE: at a 5-degree sun elevation over a NAC patch, craters cast correct shadows and astronaut tracks are identifiable at 0.5 m/px; the data HUD reads the correct ground resolution; the terminator sweep is visually convincing at every zoom; no aspect-ratio distortion at high latitude.

14.6 MARS

DATA: Caltech Murray Lab Global CTX Mosaic V01 at 5 m/px (credit NASA/JPL/MSSS/The Murray Lab); USGS MOLA DEM 463 m; USGS Viking colour 925 m; USGS MDIM 2.1 232 m; THEMIS IR day 100 m (labelled thermal); HiRISE imagery and DTMs at 0.25-0.5 m/px (credit NASA/JPL/University of Arizona).

GEOMETRY: sphere/ellipsoid, radius 3389.5 km; displacement from MOLA; HiRISE hero patches (Jezero, Gale, Valles Marineris, Olympus flank, polar layers).

MATERIALS: M1 regolithSurface with dust-tinted atmosphere (M10, Mars params).

CONTROLS: CTX luminance with Viking low-frequency colour (describe the recipe in the UI), THEMIS IR (labelled), MOLA hypsometric tint, dust opacity, polar cap season, horizon shadows, sun angle.

PACK: base 8K MDIM + 4K normal (about 35 MB); install-time 16K CTX overview + 8K MOLA + horizon map (about 480 MB); on-demand CTX hero regions and HiRISE patches.

ACCEPTANCE: rover traverse areas resolve at HiRISE scale; dunes and layered deposits show real relief; the dust parameter visibly changes the sky; colour is described as low-frequency; THEMIS is labelled thermal.

14.7 JUPITER

DATA: OPAL annual global maps (0.1 deg/px; use the highest-resolution calibrated product available to you) and/or JPL Photojournal PIA07782 (Cassini cylindrical map, NASA/JPL/SSI, lower resolution, for epochs OPAL does not cover).
GEOMETRY: ellipsoid with 1/15.4 flattening; NO displacement.
MATERIALS: M3 gasGiantSurface with real zonal wind shear; M10 for the limb.
CONTROLS: epoch selector, wind-shear speed, great red spot emphasis, aurora and lightning toggles (labelled and optional), sun angle.
PACK: base 2K + shader (about 6 MB); install-time 4K + wind LUT (about 30 MB); on-demand per-epoch maps.
ACCEPTANCE: bands shear differentially and the spot rotates with the flow; the limb is correctly dark/hazy; the HUD states the ~125 km/px ceiling and the app does not fake detail past it.

14.8 SATURN

DATA: OPAL/Cassini global maps for the body; ring optical-depth profiles from PDS (COUVIS 8001, 1 km and 10 km profiles) and PDS ring-node ancillary data.
GEOMETRY: ellipsoid with 1/10.2 flattening; ring plane annulus; correct 26.73 degree tilt.
MATERIALS: M3 body, M5 ringTransmission, M10 limb haze.
CONTROLS: optical-depth rendering, ring shadow on planet, planet shadow on rings, forward/back scatter, spokes (optional), E ring, sun angle (which changes the ring opening angle - the object's best behaviour).
PACK: base 2K body + coarse rings (about 8 MB); install-time 4K body + 16K ring profile (about 90 MB); on-demand 32K profile and E-ring haze.
ACCEPTANCE: the rings are visibly translucent where tau is small and opaque where it is large; both shadow interactions are correct at multiple sun angles; the planet is visibly oblate; the HUD states the asymmetry (rings sharper than the planet).

14.9 URANUS

DATA: OPAL maps (0.1 deg/px); Irwin et al. 2024 corrected colour; Voyager 2 archival frames for the historic appearance.
GEOMETRY: strongly oblate ellipsoid, 97.77 degree obliquity; 13 narrow rings.
MATERIALS: M3 with low-contrast banding and a methane limb; M5 for the rings.
CONTROLS: corrected vs historic appearance (both labelled); polar cap emphasis; ring set; pole-on camera preset; obliquity readout.
PACK: base 2K + rings (about 6 MB); install-time 4K + appearance pair.
ACCEPTANCE: the axial tilt is visually unmistakable; the surface is nearly featureless (this is correct, not a bug); the historic mode is labelled.

14.10 NEPTUNE

DATA: OPAL maps; Voyager 2 mosaics for the 1989 epoch; corrected colour per Irwin et al. 2024.
GEOMETRY: ellipsoid, about 1/58 flattening; Adams ring with the four arcs.
MATERIALS: M3 with epoch-bound spots; M5 for the rings.
CONTROLS: corrected vs historic appearance; epoch selector (labelled); rings and arcs; aurora toggle.
PACK: base 2K + rings (about 6 MB); install-time 4K + epochs.
ACCEPTANCE: epochs are dated in the UI; the corrected palette is the default; the deep-blue historic look is available and labelled.

14.11 MILKY WAY

DATA: NASA SVS item 4851 deep map, non-Gaia layers; USNO UCAC4 and Yale BSC5; IAU constellations and star names.
GEOMETRY: inside-out sphere (the user is inside); no displacement.

MATERIALS: unlit skybox (deep map) + M9 starSprite + constellation lines.
CONTROLS: magnitude limit slider (naked eye / binocular / telescope), deep map on/off, constellation figures, boundaries, labels, star picking.
PACK: base 4K skybox + 200k stars (about 25 MB); install-time 8K + 2M stars + constellations (about 150 MB); on-demand 16K + deeper catalogue.
ACCEPTANCE: star positions, magnitudes and colours match the catalogue within tolerances; sprites never vanish or alias when the camera rotates; zooming into a cluster resolves stars rather than blurring a texture; picking is stable.

14.12 ISS

DATA: NASA VTAD ISS_stationary.glb (44.5 MB, 247547 triangles, 28 materials, 26 PNG textures; credit NASA VTAD) and/or NASA 3D Resources ISS (B) (credit NASA/Michael D. Carbajal). Model scale must be verified against the 109 m truss span and corrected if needed.
GEOMETRY: the glTF model with 4 LODs built offline with a mesh simplifier and cache-optimised index order.
MATERIALS: M12 with clearcoat on MLI blankets, metallic Kapton foil with a high specular tint and slight anisotropy, roughened white truss paint.
CONTROLS: LOD auto/manual, sun/Earthshine/orbital-dusk lighting presets, metre scale bar, module labels, rotation hold.
PACK: base 64k-triangle LOD + half-resolution textures (about 30 MB); install-time full model + KTX2 textures (about 150 MB).
ACCEPTANCE: geometry matches the real station's proportions; MLI and foil read as different materials under the same light; no orbit or position logic exists anywhere in the code for this object (it is a standalone inspectable model, per scope); the metre scale bar is accurate.

14.13 BLACK HOLE

See section 12. **ACCEPTANCE:** physics tests pass; the photon ring is present and correctly sized; the disk's far side appears above and below the shadow; the beaming asymmetry responds to inclination; the cinematic mode is labelled as a documented artistic variant; zoom is unlimited and the structure stays correct.

=====

END OF PART II. PART III follows: performance budgets, provenance and scope enforcement, the milestone plan M-1 to M12, the prompt library, testing, the reporting protocol, and the risk register.

=====

PART III – PROCESS, MILESTONES AND PROMPTS

=====

15. PERFORMANCE BUDGETS AND INSTRUMENTATION

=====

15.1 TIERS (one enum, used everywhere)

Tier 0: 16K albedo where available, displacement on, full-resolution atmosphere and rings, TAA, optional 1.25-1.5x supersample. Target 60 fps.
Tier 1: 8K albedo, displacement on, half-resolution atmosphere upsampled. Target 60 fps.
Tier 2: 4K albedo, normal-mapped sphere (no displacement), single-scattering atmosphere. Target 30-60 fps.
Tier 3: 2K albedo, static lit sphere with normal map, no atmosphere shell. Target 30 fps.
Tier selection: automatic from a device capability probe plus the Android

thermal API, overridable in Settings. Tiers change effects only - never scientific values, never labels, never colour classifications.

15.2 BUDGETS (hold these; if a milestone cannot, report the measured number)

Resource	Tier 0	Tier 1	Tier 2	Tier 3
Frame time p95	16.6 ms	16.6 ms	33 ms	33 ms
Resident GPU textures	under 1.5 GB	under 900 MB	under 500 MB	under 250 MB
Draw calls	under 30	under 30	under 25	under 20
Triangles (focused object)	up to 4M	up to 1.5M	under 600k	under 300k
Texture uploads per frame	2	1	1	1
First pixel after tap	under 400 ms	under 400 ms	under 600 ms	under 800 ms
Full detail (hero object)	under 6 s	under 6 s	under 8 s	under 10 s

15.3 INSTRUMENTATION (build it in M1, not later)

- Frame timing via Choreographer/FrameMetrics or the engine's own timing facilities; keep a ring buffer of the last 300 frames.
- Tile store counters from §7.3.
- GPU memory estimate: sum of resident texture bytes by format and size.
- Debug overlay (debug builds only) showing fps, p50/p95, tier, resident tiles and bytes, uploads per frame, evictions, fallback frames.
- A headless benchmark mode: `--benchmark <object> --tier <n> --seconds <n>` that runs a scripted camera path and writes a CSV. This is how every performance claim in the project gets verified.

16. PROVENANCE AND SCOPE ENFORCEMENT IN CODE

16.1 WHAT THE GATE CHECKS, AND WHAT IT DELIBERATELY DOES NOT

The agent does not police licences: the owner handles licensing separately, with legal counsel, after delivery (§2.3 A1). Adding licence logic to the app or to CI would be both wasted effort and a false sense of safety. What CI does enforce is what the agent is actually responsible for:

- PROVENANCE: every shipped asset is traceable to a named product and URL.
- SCOPE: the feature stays out of the rest of ZIG (§26).

Do not add licence names, licence URLs, spdx tags or legal text to the app, the manifests or the gate. If you find yourself writing legal logic, stop: it is out of scope.

16.2 THE PROVENANCE MANIFEST IS THE SOURCE OF TRUTH

Every pack ships manifest.json (schema in Appendix A). The application reads it at runtime to build the Sources & Credits surface and the data HUD. If a manifest is missing or malformed, the pack is treated as unavailable and the app says so - it never falls back silently.

16.3 THE CI GATE (must fail the build)

tools/ci/check_provenance.py runs on every commit and must fail if:

- any asset entry is missing product, publisher, url or credit;
- an asset entry's url is not a resolvable, plausible product page (a bare domain, a search URL or a placeholder is a failure);
- a pack's declared SHA-256 does not match the built file;
- an asset is referenced by the app but has no manifest entry.

The gate prints a precise diff of what is missing and exits non-zero. It also prints the number of assets checked: ZERO ASSETS CHECKED IS A FAILURE.

16.4 THE SCOPE GATE (must fail the build)

tools/ci/check_scope.py runs on every commit in the feature branch and must fail if:

- a pre-existing file was modified and has no entry in docs/integration/CHANGED_FILES.md;
- the diff of a pre-existing file is larger than the entry's stated reason can justify (lines changed are printed for review);
- any module other than :app has gained a dependency on the museum modules, or any museum module depends on an existing feature module;
- an existing test file, string resource, asset or build configuration entry was modified at all.

It prints the changed-path list on every run, so the report never has to assert what the diff contained.

16.5 THE APP-SIDE CHECKS

- Unit test: the Sources & Credits surface renders one row per asset in the installed manifests, with the credit string verbatim, generated from the manifests and never hand-written.
- Unit test: the release manifest contains no INTERNET permission.
- Unit test: no code path references an HTTP client, WebView or any URL fetcher (assert by inspecting the release manifest and by a source scan).
- Unit test: no asset file is reachable from the app unless it has a manifest entry.

17. MILESTONE PLAN (M-1 TO M12)

=====

Each milestone: OBJECTIVE, PREREQUISITES, TASKS, DEFINITION OF DONE (DoD), and HARD STOP (the conditions under which this milestone requires a human).

The plan begins at M-1 (read-only reconnaissance of the existing repository). Do not skip it: every later milestone depends on knowing the real module names, the real navigation graph and the real Lab screen.

EXECUTION MODEL (replaces any per-session stopping rule): work the milestones in order, continuously. A milestone is complete when (a) the canonical green gate of section 23.2 passes and (b) its own Definition of Done passes. On completion, commit, tag, update PROGRESS.md, and begin the next milestone immediately. Do not wait for a human between milestones. Device-only evidence does not block advancement: it is queued in docs/verification/DEVICE_BACKLOG.md per section 24.5.

The HARD STOP list for every milestone is section 23.4; the per-milestone entries below add anything specific to that milestone. If a prerequisite milestone is not green, do not start this one - report the prerequisite failure instead.

M-1 – RECONNAISSANCE AND INTEGRATION PLAN (READ-ONLY)

OBJECTIVE: understand the existing ZIG repository well enough to integrate the feature additively, before writing a single line of production code. PREREQUISITES: none. This is the first milestone of the plan.

TASKS:

1. Read the repository: settings.gradle(.kts), every module build file, the version catalog, the Android manifest(s), the navigation graph, the design system, the Lab screen and its neighbours, string resources, the test setup, and the CI workflows. Record what is ACTUALLY there, with file paths.
2. Write docs/integration/INVENTORY.md: modules, package roots, SDK levels, Kotlin/Compose/AGP/JDK versions in use, DI approach, navigation approach, theming, resource conventions, test frameworks, CI jobs. Every line must be traceable to a file you read; no invented entries.
3. Write docs/integration/CHANGED_FILES.md: the pre-existing files you expect this feature will have to touch, one line each, with the reason and why the change is unavoidable (§4.5). Keep it current at every milestone. New files the feature adds do not belong in it.
4. Write docs/integration/COMPONENT_REUSE.md: the existing components, colours,

typography and icons the new card and the museum grid will use.

5. Write docs/SPEC_MUSEUM.md: the Space Museum feature as it will exist in THIS repository - module names, package names, screen structure, navigation entries, and the Lab screen card's insertion point.
6. Write docs/integration/ENVIRONMENT.md: what the local build environment needs (JDK, SDK, Gradle, plus Python/GDAL/KTX-Software for the asset kit) and how that differs from §3.
7. Create docs/SOURCES.md: the source index. For every asset the feature will use, the product name, the publisher and the URL it will be fetched from (A2). This is traceability, not a licence record: the owner clears rights separately, afterwards. Extend it as assets are added.
8. Start docs/decisions/DECISIONS.md with: repository-wins-over-brief differences, module naming, and the integration strategy.
9. NO PRODUCTION CODE CHANGES IN THIS MILESTONE. Reconnaissance is read-only.

DoD:

- All documents exist, and every path and symbol in them was read from the repository: no document may contain a path that does not exist.
- CHANGED_FILES.md contains no entry whose reason does not survive the test in §26.7 (does the feature fail to work without it?).
- The report states, in numbers: module count, existing test count and their pass state, CI job count, and the app's real toolchain versions against §3.

HARD STOP (23.4 plus these): the repository cannot be read; the Lab screen cannot be located; the app's structure makes an additive integration impossible.

M0 – MODULE SCAFFOLD, CARD, NAVIGATION

OBJECTIVE: the Space Museum card exists in the Lab screen, opens an empty museum screen, and ZIG still builds, installs and behaves exactly as it did before. PREREQUISITES: M-1.

TASKS:

1. Create the new Gradle modules using the exact names recorded in docs/SPEC_MUSEUM.md, registered with the minimum include() lines in settings.gradle(.kts). New modules only; no existing module is restructured.
2. Implement :core:model with ObjectSpec and the thirteen registry entries (name, radius, oblateness, tilt, rotation period, pack id, ceiling text). Comment the source of every numeric value. This is a NEW module: ZIG's existing models are not touched, renamed or reused.
3. Implement the museum grid screen (thirteen tiles, name plus ceiling subtitle) reusing the app's existing design system, and an empty viewer screen behind it inside the new feature.
4. Add the ONE card to the Lab screen and the ONE navigation destination, in the style that file already uses, and record both in docs/integration/CHANGED_FILES.md. Nothing else in those files changes (section 26.5).
5. Implement :tools:ci/check_provenance.py and :tools:ci/check_scope.py against the §16.3 and §16.4 rules, wire both into Gradle and CI, and commit deliberately broken fixtures: each gate must fail on its fixture and pass on the valid case.
6. Implement :core:credits with the manifest reader and an empty credits screen.
7. Configure the new modules' release build so no new INTERNET permission, component or manifest attribute is introduced anywhere in the merged manifest.
8. Run ZIG's ENTIRE existing test suite, unmodified, and record the result.

DoD:

- ./gradlew assembleDebug succeeds; the Lab screen shows one new card and nothing else has moved; tapping it opens the museum grid; all thirteen tiles are present; system back returns to the Lab screen.

- The provenance gate fails on the broken fixture and passes on the valid one, and prints the asset count it checked; the scope gate reports the changed-path list.
- ZIG's pre-existing tests all pass, unmodified (evidence: the same command run before and after).
- Every pre-existing file changed by this milestone appears in CHANGED_FILES.md, with the diff small enough that the stated reason explains it (G10).
- apkanalyzer output shows no INTERNET permission in the release variant.
- No TODO placeholders in :core:model or :core:credits.

HARD STOP (23.4 plus these): the Lab screen cannot accept a card without changing existing behaviour; ZIG's existing tests already fail for reasons unrelated to this work and cannot be left failing.

M1 – ENGINE CORE, HDR PIPELINE, CAMERA, FIRST OBJECT ON SCREEN

OBJECTIVE: one hard-coded object (a generated ellipsoid with a bundled test texture) rendering correctly with the full pipeline and controls. PREREQUISITES: M0. TASKS:

1. InspectorEngine in :core:engine: Engine lifecycle, scene, view, camera, renderer, and the frame loop. No leaks: implement release paths in the same commit.
2. Camera rig per §5.4 (orbit, damping, pinch zoom, tap-to-focus stub, double-tap reset, near/far from zoom).
3. Unit-radius normalisation per §5.3 and the registry-driven ellipsoid generation with oblateness.
4. HDR pipeline: half-float target, bloom, tone mapper with a runtime switch between AgX and PBR Neutral, TAA, dithering. Record the exact API names used in DECISIONS.md.
5. Directional sun light with the physical scale factor and an EV exposure control wired to a slider.
6. Instrumentation from §15.3 including the debug overlay.
7. A first material (M1 skeleton: albedo + normal, no displacement yet), built offline with matc and loaded from assets.
8. A Compose surface that hosts the view and survives configuration changes.

DoD:

- A textured ellipsoid orbits smoothly; pinch zoom works from full disk to surface; no jank (report p50/p95 frame times at tier 0 on the reference device).
- The debug overlay reports fps, tier, and zero leaked resources after ten object re-creations (add a debug action that recreates the object).
- Tone mapper switch visibly changes the image; TAA on/off toggles.
- Instrumentation CSV is produced by the benchmark mode.

HARD STOP (23.4 plus these): Filament cannot be initialised on the reference device; the HDR target is unsupported on the minimum device (report device model and caps).

M2 – ASSET KIT (OFFLINE PIPELINE)

OBJECTIVE: the offline toolchain that turns a source raster into a verifiable pack, demonstrated end to end on one small real dataset. PREREQUISITES: M0 (python/GDAL/KTX-Software available on the build machine).

TASKS:

1. Implement the `assetkit` CLI verbs from §6.2 with --dry-run on every verb.
2. Implement reprojection to equirectangular, high-quality downsample, mip generation in linear space (§6.5), and the tile pyramid with apron and the longitude seam duplication rule (§6.4).

3. Implement `assetkit horizon`, `assetkit normal`, `assetkit encode`, `assetkit pack`, `assetkit verify`.
4. Implement manifest generation (Appendix A) including SHA-256 of every file and the real ground resolution of each map.
5. Demonstrate on a SMALL real source: a crop of the USGS lunar DEM and a crop of a public-domain Moon mosaic (a few hundred MB maximum). Produce a pack, verify it, and load it on device in a throwaway debug screen.
6. docs/pipeline.md: the exact command sequence to rebuild every pack, with the source product and URL for each input (and docs/SOURCES.md updated to match).

DoD:

- Determinism: running the pipeline twice produces byte-identical packs (prove by hashing both).
- A device debug screen displays the pack's texture with correct colour space (no washed-out or dark image) and the manifest's ground resolution shown.
- `assetkit verify` fails on a deliberately corrupted tile.

HARD STOP (23.4 plus these): a source dataset cannot be downloaded, or its product page cannot be located - stop and report; do not substitute a different dataset silently.

M3 – TILE STORE (THE CRITICAL PATH)

OBJECTIVE: inspect a >8192 axis-width pyramid smoothly, with no frame stalls.

PREREQUISITES: M1, M2.

TASKS:

1. Implement TileStore per §7 (structures, level selection, worker decode, upload budget, LRU eviction, prefetch along motion, fallback to coarser levels).
2. Read pyramids from a pack via ZipFile with no per-file recompression.
3. Integrate level selection with the camera and with the data HUD.
4. Evaluate Filament's virtual texturing against your tile store and pick one; record the comparison numbers in DECISIONS.md.
5. Stress test: a synthetic 32768-wide pyramid, scripted camera path from full disk to surface over 60 seconds, at tier 0 and tier 2.

DoD:

- During the entire stress path, no frame exceeds 33 ms (report max frame time and the p95).
- Resident bytes never exceed the tier budget; eviction counters increment.
- Visual: no seams between tiles at any zoom (verify by a screenshot sweep at 1.0x, 2x, 4x, 8x with the camera panning across a tile boundary).
- The HUD's displayed resolution tracks the level actually resident.

HARD STOP (23.4 plus these): the stress test cannot meet the frame budget after optimisation

- report measurements, then implement the documented fallback (discrete LOD maps per object, no virtual texturing) as a separate, explicit decision.

M4 – MATERIAL LIBRARY

OBJECTIVE: all thirteen materials from §8 compiled, loadable, and visually verified in isolation. PREREQUISITES: M1 (M2 for texture inputs).

TASKS:

1. Author M1-M13 .mat sources with parameters as specified, and a matc build step in Gradle.
2. Implement M1 fully: displacement in the vertex block, normal and height sampling, horizon-map shadows, A0, procedural toggle.
3. Implement M3, M4, M5, M6, M7, M8, M9, M10 shells, M12, M13. M11 is a stub in this milestone (structure only).
4. Add a material testbed screen (debug only) that renders each material on a

sphere with sliders for every parameter.

5. Golden-image tests per material: a fixed camera, fixed light, fixed exposure, rendered offscreen, compared against a committed reference with a stated tolerance.

DoD:

- Every material loads with no warnings; no shader compilation at runtime.
- Golden images pass on the reference device within tolerance.
- M1's horizon shadow produces a visible, correct shadow at a 5-degree sun elevation on a displaced sphere built from the lunar DEM crop.
- Tier variants exist for M1, M3, M10 and compile.

HARD STOP (23.4 plus these): a required engine feature is not exposed in the pinned version (for example a specific sampler type or a post effect) - report and propose the equivalent custom material rather than abandoning the requirement.

M5 – CONTROLS, DATA HUD, CREDITS

OBJECTIVE: the product's interaction model complete on the first real object.

PREREQUISITES: M1, M4.

TASKS:

1. Rotation control (real sidereal period; 1x/60x/3600x/hold) driven purely by the control, never wall clock.
2. Sun-direction control (azimuth, elevation, EV) with the pure function from §10.3 and its unit tests; per-object presets stored as data.
3. Layer toggles per object (data-driven from the registry).
4. Data HUD and procedural-detail indicator per §5.9, driven by the object's declared ceiling and the resident tile level.
5. Sources & Credits surface generated from the installed manifests; unit tests for verbatim credit strings.
6. Camera presets per object (full disk, pole-on, terminator, hero region).
7. Tap-to-focus with analytic surface intersection and a smooth recentre.

DoD:

- Every control is exercised by an automated UI test (Compose test rules).
- Rotating the object and moving the sun both leave all labels and HUD values consistent; no effect depends on wall-clock time (prove by a test that advances a fake clock and shows the render is unchanged).
- The Sources & Credits surface lists exactly the assets of the installed packs.

HARD STOP (23.4 plus these): none expected; report if a Compose test cannot be made stable.

M6 – OBJECTS: MOON, MARS, MERCURY (the rocky trio)

OBJECTIVE: three objects shipping at maximum fidelity with verified ceilings.

PREREQUISITES: M2, M3, M4, M5.

TASKS:

1. For each object: fetch the specified sources, build base / install-time / on-demand packs, write manifests, wire the registry entries and the layer toggles, verify the ceiling text.
2. Build the horizon maps and normal maps offline from the DEMs.
3. Curate hero patches: 20 lunar NAC sites, 20 Mars CTX regions and 50-100 HiRISE patches, one Mercury regional DTM where available.
4. Per-object acceptance run from §14 and a screenshot set committed to tests/golden/<object>/.

DoD:

- All three objects pass their §14 acceptance criteria.
- The provenance gate passes with the real manifests; every asset names a real

product and a resolvable URL.

- The data HUD reports the correct ground resolution per map, verified against the manifest.
- Memory and frame budgets met at tier 0 and tier 2 on the reference device.

HARD STOP (23.4 plus these): a specified dataset is unavailable, or the only source found is not the product this brief describes - stop and report what you actually found.

M7 – OBJECTS: EARTH, VENUS, SUN

OBJECTIVE: the atmosphere-heavy objects. PREREQUISITES: M6 (reuse the pipeline), M4's M10.

TASKS:

1. Build the offline atmosphere LUT generator and the per-body parameter sets (Earth with ozone, Venus Mie-dominated, Mars dust, gas-giant methane).
2. Earth: BMNG months, night lights, cloud layer, ocean roughness and glint, bathymetry, Landsat hero tiles.
3. Venus: cloud deck default, radar/topography/Fresnel/slope modes, peel transition.
4. Sun: M6 and M7 materials, AIA/HMI channels with labels, magnetogram mode, corona control, granulation animation in the shader's third dimension.

DoD:

- Earth's sunset band, blue limb, glint, and night-light behaviour pass the stated acceptance tests.
- Venus hides its surface in cloud mode and is labelled in radar mode.
- The Sun shows stable granulation, correct limb darkening, and no flat yellow texture; every EUV mode is labelled false colour.
- Golden images committed.

HARD STOP (23.4 plus these): the atmosphere LUT generator cannot be implemented from published equations - stop and report; do not vendor third-party shader source (A4).

M8 – OBJECTS: JUPITER, SATURN, URANUS, NEPTUNE

OBJECTIVE: the giants and the ring system.

PREREQUISITES: M6, M7 (for M10 reuse).

TASKS:

1. Giant-planet material: wind LUT ingestion, shear, methane limb, oblateness.
2. Ring system: extract tau profiles from the PDS products, build the radial (and azimuthal) ring textures, implement M5 with transmission, phase asymmetry, and both shadow directions.
3. Uranus and Neptune: appearance modes (corrected and historic, both labelled), epoch selector for Neptune, narrow ring sets.
4. Jupiter: epoch selector, wind shear, aurora and lightning toggles (optional, labelled).

DoD:

- Saturn's ring transmission and both shadow interactions pass their acceptance tests at three sun geometries.
- The giants are visibly oblate and their limbs are correctly hazy.
- Histories are labelled; no unlabelled "current" implication anywhere.
- Ring textures are as sharp as the 1-10 km PDS profiles allow; the UI states the asymmetry.

HARD STOP (23.4 plus these): PDS ring products cannot be parsed into a radial profile with the documented tools - report the obstacle; do not substitute an artistic ring texture.

M9 – OBJECTS: MILKY WAY AND ISS

OBJECTIVE: the two "different" objects.

PREREQUISITES: M4 (M9 starSprite, M12), M5.

TASKS:

1. Catalogue ingestion: read UCAC4 and Yale BSC5 into the compact binary VBO format; validate a sample against published values (unit tests for a handful of well-known stars: position, magnitude, colour index).
2. Sprite rendering with flux-preserving sizing and a minimum-size clamp; magnitude-limit slider; constellation figures, boundaries and names; star picking with a stable hit test.
3. Deep map as skybox; verify no Gaia-derived layer is used; record the exact file.
4. ISS: build 4 LODs offline, convert textures to KTX2, author M12 variants (clearcoat MLI, Kapton foil, white paint), metre scale bar, module labels, lighting presets.

DoD:

- Star catalogue validation passes for at least 20 named stars.
- Rotating the camera at high zoom produces no sprite flicker or disappearance (a scripted 60-second rotation test with a frame-by-frame flux check).
- Zooming into a cluster resolves stars; the deep map does not blur into visible texels (screenshot evidence).
- The ISS model's truss measures 109 m within tolerance; the three material families are visually distinct under the same light.

HARD STOP (23.4 plus these): the chosen SVS file cannot be shipped without Gaia-derived content - stop and report; choose an alternative layer or source.

M10 – BLACK HOLE

OBJECTIVE: the highest-risk object, complete and validated.

PREREQUISITES: M4 (M11), M5 (slider UI).

TASKS:

1. :tools:blackhole-lut: generate D(e,u) and U(e,phi) tables plus the blackbody colour table, with a --verify verb comparing table results against direct numerical integration, printing the maximum error.
2. M11 lensing material in the skybox domain, sampling the deep map with deflected directions.
3. Disk geometry and material: temperature profile, blackbody colour lookup, Doppler and gravitational shift with a single stated intensity convention.
4. Tiers and the user-facing quality slider with named presets.
5. Modes: physically correct, cinematic (labelled), M87-like, Sgr A-like.
6. Validation against an independent CPU reference for three configurations; commit the comparison images and the measured tolerances.

DoD:

- Unit tests for b_c and the shadow radius pass.
- The LUT verify verb reports a maximum error under the stated tolerance.
- The reference comparison passes for all three configurations (photon-ring radius, far-side arc, brightness ratio).
- Tier 2 holds 30 fps on a mid-range device and tier 0 holds 60 fps on the reference flagship; both measured with the benchmark mode.

HARD STOP (23.4 plus these): the tables cannot reach the tolerance - report the measured error, then fall back to a higher-resolution table rather than loosening the tolerance silently.

M11 – PERFORMANCE, THERMALS, DEVICE MATRIX

OBJECTIVE: hold the budgets on real devices, sustained.

PREREQUISITES: M6-M10.

TASKS:

1. Implement automatic tier selection from device capability and thermal status, with a Settings override.
2. Run the benchmark path for every object at every tier: produce a CSV of frame times, resident bytes, and peak temperatures where available.
3. Fix all budget violations; where a fix is impossible, record the exception in DECISIONS.md with measurements.
4. Device matrix: minimum two Adreno, two Mali, one Xclipse class device if obtainable; record driver strings and any artifacts.

DoD:

- Ten-minute session at tier 0 per hero object with no thermal collapse below the tier floor; p95 within budget; no crashes; no leaked resources.
- The CSV and the matrix results are committed under docs/perf/.

HARD STOP (23.4 plus these): a device-specific driver bug cannot be worked around - document it, add a device-specific tier cap, and report.

M12 – POLISH, ACCESSIBILITY, RELEASE PREP

OBJECTIVE: shippable. PREREQUISITES: M11.

TASKS:

1. Accessibility: content descriptions for every control, scalable text, contrast checks, and a reduced-motion mode.
2. Localisation scaffolding (English complete; strings externalised).
3. Deep links / shareable state ("show me Apollo 17", "Sun in 304").
4. Store metadata: description that states the offline nature and the data provenance; screenshots per object at tier 0.
5. Final provenance pass: Sources & Credits complete for every shipped asset, docs/SOURCES.md matching the manifests, and both gates green. Confirm no third-party source code was vendored anywhere in the tree (A4).
6. Final acceptance checklist (section 22) executed and reported item by item.

DoD:

- The final checklist passes with no exceptions other than those explicitly waived and recorded.
- A release AAB builds with the correct pack configuration and installs cleanly on a device with no packs available (all thirteen buttons functional).

18. PROMPT LIBRARY

Part 18 is the prompt library. Two things changed in revision 1.1: the library no longer instructs the agent to stop between milestones, and it now tells the agent how to work without a local emulator or phone (section 24). The STANDING PROMPT applies to every working session; the milestone prompt template (18.2) frames each milestone; 18.3 is a fully worked example. Because execution is continuous, the agent reads the standing prompt once per session and then works milestones in sequence without further prompting.

18.1 STANDING PROMPT (paste first, every session)

You are adding a new feature called SPACE MUSEUM to ZIG, an existing Android astronomy app in Kotlin and Jetpack Compose, using Google Filament for the rendering. The specification is the PDF "Space_Museum_Roadmap" in the repository root (or attached to this conversation). The research background is the companion PDF on NASA-Eyes-level local rendering.

Standing rules, which override any instruction that conflicts with them:

1. Work the milestone plan in order and CONTINUOUSLY. After a milestone passes

- the green gate (23.2) and its Definition of Done, commit, tag, update PROGRESS.md and start the next milestone immediately. Do not wait for a human between milestones. Stop only for the hard stop conditions in 23.4.
- Read Part I of the brief in full, then only the sections the milestone cites.
 - Never invent assets, URLs, product names, API names or scientific values. If an input is unavailable or unverifiable, stop and report.
 - Respect the scope lock: one object on screen, no orbits, no ephemerides, no network in the runtime, no WebView, no multi-object scenes.
 - Touch nothing irrelevant: apply the necessity test of section 26.7 to every edit. Licensing is not your concern: the owner handles it after delivery (2.3 A1). Do not paste or vendor third-party source code (A4), and do not add licence text or legal logic to the app or to CI.
 - Everything ships offline: no runtime downloads, no analytics, no telemetry, and no INTERNET permission in the release manifest.
 - Quality over package size, but never at the cost of scientific honesty: if the app invents detail beyond the real data, it must label it.
 - Every GPU resource you create must be released on switch, pause and destroy, in the same commit.
 - Keep the architecture of section 4: :core:engine owns all Filament types, :core:model is pure Kotlin, features talk to the engine through interfaces.
 - Record every non-obvious decision in docs/decisions/DECISIONS.md with the reason and, where relevant, the measurement.
 - Run the green gate (23.2) before advancing: build both variants, unit tests, lint, the provenance and scope gates, and the offline asset verification suite. A milestone is not done until its gate and its Definition of Done both pass.
 - Verify what you can without a device (section 24). Use the CI emulator for interaction and smoke tests; queue device-only evidence in docs/verification/DEVICE_BACKLOG.md rather than blocking.
 - Never claim a measurement you did not make (24.7). Use TARGET or ESTIMATE for anything not produced by a real device.
 - Report in the format of section 20. Do not pad the report; be specific.

If any part of the milestone is impossible as specified, say so explicitly, explain what you tried, and propose the closest compliant alternative. Never silently substitute.

18.2 MILESTONE PROMPT TEMPLATE

Milestone: M<N> - <TITLE>

Objective: <copy from section 17> Read: <the sections cited by the milestone> Tasks: <the numbered tasks from section 17> Definition of Done: <the DoD from section 17> Stop conditions: <from section 17>

Deliver, in this order:

- A short plan (what files, what order, what you will verify).
- The implementation.
- Evidence: commands run and their output; gate results; screenshots or measurement CSVs where the DoD requires them; the evidence path under docs/verification/M<N>/.
- A definition-of-done checklist with each item marked pass/fail and why, and a separate list of anything deferred to the device backlog.
- Then, if the gate is green: commit, tag m<N>-green, update PROGRESS.md, and PROCEED to milestone M<N+1> without waiting for a human. Only stop if a hard stop condition (23.4) applies - in that case, state it and halt.

18.3 EXAMPLE: MILESTONE M3 PROMPT (fully worked, as a model)

Milestone: M3 - TILE STORE (the critical path) Objective: inspect a pyramid whose axis width exceeds 8192 smoothly, with no frame stalls, on the reference device.

Read: brief section 7 (tile store), 15.2-15.3 (budgets and instrumentation),
[5.9 \(data HUD\)](#), [6.4 \(tile rules\)](#), and [section 4 for module rules](#).

Prerequisites to confirm before starting: M1 (engine core, instrumentation, camera) and M2 (asset kit producing a pyramid) are both complete and their DoDs pass. If either is not, stop and report.

Tasks:

1. Implement TileKey, TileEntry, TileStore in :core:data with the state machine REQUESTED/DECODING/RESIDENT/EVICTABLE.
2. Implement level selection from projected texel density, with unit tests for three cameras (full disk, half zoom, surface) using hand-computed expected levels.
3. Implement worker decode (3 threads, IO dispatcher), upload budget (2 textures per frame at tier 0, 1 otherwise), LRU eviction with a protected visible set, prefetch along the camera motion vector, and fallback to coarser resident levels.
4. Wire resident level to the data HUD.
5. Evaluate Filament virtual texturing against this implementation and record the comparison (quality, memory, code size, frame behaviour) in DECISIONS.md.
6. Build the stress harness: a synthetic 32768-wide pyramid and a 60-second scripted camera path from full disk to surface, at tier 0 and tier 2, emitting the CSV.

Definition of Done:

- No frame exceeds 33 ms during the entire stress path; report max and p95.
- Resident bytes never exceed the tier budget; eviction counters increment.
- No tile seams at 1x/2x/4x/8x while panning across a boundary (screenshots).
- The HUD's resolution readout matches the resident level.
- The provenance and scope gates, and the full test suite, pass.

Deliver the five items from the template. If the gate is green, tag m3-green and continue directly to M4; if it is red, fix it and re-run, or stop under 23.4 if the cause is outside your control. Device-dependent parts of the DoD (real-GPU frame times, thermal behaviour) are queued in DEVICE_BACKLOG.md, not treated as failures.

[18.4 MILESTONE PROMPTS M-1 TO M12 \(condensed; expand each with the 18.2 template\).](#)

Each of these is executed in sequence without pausing for a human. The condensed form below is enough when the milestone is unambiguous; expand to the full 18.2 template when a milestone is complex (M2, M3, M4, M6, M7, M10) or when a gate fails and you need to restate scope precisely.

- M-1 Reconnaissance, READ-ONLY: read the repository, find the Lab screen, write INVENTORY.md, SPEC_MUSEUM.md, CHANGED_FILES.md, COMPONENT_REUSE.md, ENVIRONMENT.md, SOURCES.md and the first DECISIONS.md entries. No production code. Record the real toolchain against section 3, and the real path of the Lab screen and how its cards are declared.
- M0 Scaffold: new modules, thirteen registry entries with cited sources, museum grid, the card and its destination added the way the Lab screen already does it, provenance and scope gates with failing fixtures, Sources & Credits reader, DECISIONS.md, no INTERNET permission in release, ZIG's own tests run unmodified and green.
- M1 Engine core: lifecycle, camera rig, unit-radius normalisation, HDR pipeline with AgX/PBR Neutral switch, bloom, TAA, physical sun with EV control, instrumentation and benchmark mode, first material, Compose surface.
- M2 Asset kit: the nine CLI verbs, reprojection, linear-space mips, tile pyramid with apron and seam duplication, horizon and normal maps, manifest

generation, determinism proof, one real small pack loaded on device.

- M3 Tile store: see 18.3.
- M4 Materials: M1 fully with displacement and horizon shadows, M3-M13 authored, material testbed, golden-image tests, tier variants.
- M5 Controls and HUD: rotation, sun direction with unit tests, layer toggles, data HUD and procedural indicator, credits screen, camera presets, tap-to-focus.
- M6 Rocky trio: Moon, Mars, Mercury - packs, horizon maps, hero patches, per-object acceptance, golden screenshots.
- M7 Earth, Venus, Sun: atmosphere LUT pipeline and parameter sets, Earth layers and Landsat tiles, Venus modes, Sun channels and corona.
- M8 Giants and rings: wind shear, PDS tau profiles, ring material with both shadow directions, appearance modes and epochs.
- M9 Milky Way and ISS: catalogue ingestion with validation, sprite rendering with flux preservation, deep map provenance check, ISS LODs and materials.
- M10 Black hole: LUT generator with verification, lensing material, disk with beaming, tiers and slider, four modes, CPU reference comparison.
- M11 Performance: automatic tiers, thermal handling, benchmark CSV per object and tier, device matrix, budget violations fixed or recorded.
- M12 Polish: accessibility, localisation scaffolding, deep links, store metadata, final legal pass, final checklist.

19. TESTING STRATEGY

19.1 UNIT TESTS (JVM, fast, in CI)

- Camera maths, sun-direction function, unit-radius conversion, registry values.
- Tile level selection with hand-computed expectations.
- Manifest parsing and the provenance-gate rules (including the negative fixtures).
- Star catalogue sample values against published data.
- Black-hole physics: b_c, shadow radius, table verification tolerance.

19.2 GOLDEN-IMAGE TESTS (device, tolerance-based)

- Fixed camera, fixed sun direction, fixed exposure, offscreen render, compared to a committed reference with a stated per-pixel tolerance and an SSIM floor.
- One golden per material and per object's canonical view; regenerate only with an explicit decision entry explaining the visual change.
- Pin the reference device; document its GPU and driver. Golden tests on a different GPU are informational, not gating.

19.3 INTERACTION TESTS (Compose UI tests)

- All thirteen buttons; every toggle; the rotation and sun controls; the data HUD text; the credits list contents; the procedural indicator appearing past the ceiling.

19.4 STRESS AND SOAK

- Scripted camera paths per object at each tier, 10 minutes, with the CSV.
- Rapid object switching (100 switches) with a leak assertion on texture and renderable counts.

19.5 REGRESSION RULES

- Any bug fix that touches a material, the tile store or the camera adds a test that would have caught it.

20. REPORTING PROTOCOL (exactly this format, no padding)

1. MILESTONE AND STATUS: one line, e.g. "M3 COMPLETE" / "M3 BLOCKED".
2. PLAN AS EXECUTED: bullet list of what was actually done (not a promise).

3. FILES: added / changed / removed, with paths.
4. EVIDENCE: commands run and their relevant output; test counts; screenshots or CSV paths; measurements with units. Paste the numbers, not adjectives.
5. DEFINITION OF DONE: each item, marked pass or fail, with the evidence reference.
6. DEVIATIONS: anything done differently from the brief, and why, with the DECISIONS.md entry name.
7. BLOCKERS: what is blocked, by what, and precisely what you need.
8. NEXT: the milestone started in the same session (the continuous model of RULE 1); if this report ends the plan, the first item of the section 25 handover.
9. SYNC: pushes made this milestone, throttle events and retries with timestamps, and the CI state (§28.7). Keep it to one line when nothing happened.

21. RISK REGISTER AND FALLBACKS

Risk	Impact	Mitigation	Fallback
Tile streaming misses the frame budget	Product-defining	Build it early (M3); instrument; prefetch; upload budget	Discrete LOD maps per object (still better than any competitor); documented decision
Vertex-shader displacement too slow on target GPUs	Relief quality	Measure in M4 with the lunar DEM crop; LOD bias	Pre-baked 4-level displaced mesh chain
Atmosphere LUT cannot be implemented from published equations	Earth/Venus/Mars realism	Implement from the papers (A4)	Single-scattering only, plus a cheap analytic limb; label the tier
Black-hole tables miss tolerance	Flagship object	Higher-resolution tables; better integration	Analytic strong-deflection approximation plus a baked deflection field; document accuracy
A specified dataset is unavailable or has moved	Scope/schedule	Verify at fetch time; provenance gate; docs/SOURCES.md	Use the next-best entry from §27.4 and report it; never substitute silently
PDS ring products hard to parse	Saturn	Use the documented profile products and their readers	Reduced-fidelity radial profile from published ring tables; label the resolution honestly
Device driver artifacts on one GPU family	Stability	Device matrix; tier caps	Per-device tier cap recorded in DECISIONS.md
Thermal throttling in long sessions	UX	Thermal API; tier stepping	Frame pacing and an explicit "cooling" state message
Scope creep back toward a simulator	Product	Scope lock in the brief; RULE 1 in every prompt	Reject with a scope-lock citation; propose a separate mode in a future release

22. FINAL ACCEPTANCE CHECKLIST

Deliverable checks (all must pass before release):

- [] All thirteen buttons open a correct, correctly labelled render, offline, with no packs installed (base module only).
- [] With maximum packs installed, each object reaches its declared data ceiling and the HUD states the ground resolution at every zoom level.
- [] The procedural-detail indicator appears exactly past each object's ceiling and the procedural layer can be switched off.
- [] Rotation uses the object's real sidereal period; the sun-direction control drives terminator, glint, ring shadows and ring opening angle consistently.
- [] No screen in the app requires or performs a network request; the release manifest contains no INTERNET permission.
- [] The Sources & Credits surface lists every asset of every installed pack with the exact credit string, generated from the manifests.
- [] The provenance and scope gates pass, and docs/SOURCES.md matches the shipped manifests asset for asset.

```
[ ] Golden-image tests pass for every material and every object's canonical view
    on the reference device.
[ ] Frame budgets hold per tier for ten-minute sessions; no leaks after 100
    object switches.
[ ] Every object's acceptance criteria from section 14 pass, with evidence
    committed under tests/golden and docs/perf.
[ ] docs/pipeline.md can rebuild every pack from scratch, byte-identically.
[ ] DECISIONS.md contains an entry for every deviation and every non-obvious
    choice, with measurements where applicable.
INTEGRATION CHECKS (revision 3.0; all must pass):
[ ] Every pre-existing file changed by the whole feature is listed in
    docs/integration/CHANGED_FILES.md with a necessity that survives the §26.7
    test. Proven by diff, not by assertion.
[ ] ZIG's existing test suite passes unmodified; no existing test expectation was
    edited to accommodate the new feature.
[ ] The Lab screen before/after screenshots differ by exactly one added card.
[ ] No existing calculation, screen, string, permission or dependency changed,
    and no museum module is referenced by any existing ZIG code except :app.
[ ] The ISS museum model is referenced by nothing in ZIG except the museum's own
    viewer; the app's ISS calculations produce identical results before and after.
ASSET CHECKS (revision 2.0):
[ ] Every shipped asset traces to a product, publisher and URL in its manifest
    and in docs/SOURCES.md, so the owner can clear rights afterwards.
[ ] No third-party source code was pasted or vendored anywhere in the tree (A4);
    external code exists only as declared, pinned dependencies.
[ ] Nothing in the app or in CI contains licence logic, licence text or legal
    claims: licensing is out of scope and is handled outside the repository.
GITHUB CHECKS (revision 2.0):
[ ] Push count over the whole build stayed within the §28.2 budget, and every
    throttle event is recorded in PROGRESS.md with its retry outcome.
```

```
=====
END OF PART III.
=====
```

```
=====
APPENDICES
=====
```

```
=====
APPENDIX A – PACK MANIFEST SCHEMA (manifest.json)
=====
```

```
{
  "schemaVersion": 1,
  "object": {
    "id": "moon",
    "displayName": "Moon",
    "sceneRadiusMetres": 1737400.0,
    "oblateness": 0.0,
    "axialTiltDeg": 6.68,
    "rotationPeriodHours": 655.728,
    "dataCeilingText": "0.5 m/px at Apollo sites | 100 m/px global",
    "ceiling": { "albedoMetresPerTexel": 100.0, "reliefMetresPerTexel": 474.0,
      "sourceNote": "LROC WAC morphology mosaic; LOLA ldem_64" }
  },
  "pack": { "id": "moon", "tier": "install-time", "builtAt": "2026-09-14T00:00:00Z",
```

```

    "builderVersion": "assetkit 1.0.0", "totalBytes": 545000000 },
"assets": [
  {
    "id": "moon.wac.16k",
    "role": "albedo",
    "file": "maps/albedo_16k.ktx2",
    "sha256": "<hex>",
    "bytes": 59600000,
    "texture": { "width": 16384, "height": 8192, "format": "ASTC_6x6",
      "transfer": "srgb", "mips": true },
    "source": {
      "product": "Moon LRO LROC WAC Global Morphology Mosaic 100m",
      "publisher": "USGS Astrogeology Science Center / NASA GSFC / ASU",
      "url": "https://astrogeology.usgs.gov/search/map/moon_lro_lroc_wac_global_morphology_mosaic_100m",
      "credit": "NASA/GSFC/Arizona State University; mosaic published by USGS Astrogeology",
      "retrievedAt": "2026-09-14",
      "groundResolutionMetresPerTexel": 100.0,
      "meaning": "true colour morphology, photometrically normalised",
      "processing": ["reprojected to equirectangular",
        "Lanczos downsample 16k x 8k",
        "mips in linear space", "ASTC 6x6"]
    }
  },
  {
    "id": "horizon", "file": "maps/horizon_8k.ktx2", "role": "horizonShadow" } ],
"tiers": { "0": ["moon.wac.16k"], "1": ["moon.wac.8k"], "2": ["moon.wac.4k"] },
"notes": ["Polar rows oversampled; see pipeline.md section 4."]
}

```

Rules: every asset entry MUST have product, publisher, url and credit. The CI gate enforces this. Missing any field makes the pack unusable for that object, and the app must say so rather than fall back silently.

There is deliberately no licence field. Licensing is handled by the owner outside this repository, after delivery (\$2.3 A1); the manifest exists so that every shipped file can be traced back to a named product and page. Do not add licence names, SPDX tags, licence URLs or legal text to this schema.

APPENDIX B – TEXTURE AND FORMAT QUICK REFERENCE

Purpose	Format	Block/quality	Notes
Albedo (science)	R8G8B8_SRGB + ASTC	6x6	4x4 for hero objects
Albedo (hero patch)	R8G8B8_SRGB + ASTC	4x4	largest data, smallest area
Normal	R8G8_UNORM + UASTC	quality 4	verify ASTC 6x6 visually first
Height	R16_SFLOAT or R8_UNORM + ASTC	6x6	R16F when metres are needed
A0 / masks	R8_UNORM + ASTC	6x6	separate texture from albedo
Horizon (8 dirs)	R8_UNORM + ASTC	6x6	or 2 x RGBA8 for 16 dirs
Night lights	R8G8B8_SRGB + ASTC	6x6	additive layer
HDR sky map	R11F_G11F_B10F or RGB9_E5	n/a	ASTC HDR is not broadly supported
Lensing LUTs	R32G32_SFLOAT	n/a	RG32F, no mips
Atmosphere LUTs	R16_SFLOAT	n/a	no mips; clamp to edge
Blackbody table	RGBA16F	n/a	1D as 256x1

Memory formula: bytes ≈ (W x H x bitsPerTexel) / 8, then add about 33% for mips.
 ASTC: 4x4 = 8.0 bits/texel, 6x6 = 3.56, 8x8 = 2.0.

APPENDIX C – ATMOSPHERE PARAMETER SETS (fields to expose per body)

Fields (all in SI or per-metre where a length is implied):

- rayleighScatteringRGB (per metre, at 3 reference wavelengths)
- rayleighScaleHeightMetres
- mieScatteringScalar, mieExtinctionScalar, mieScaleHeightMetres, mieAsymmetryG
- absorptionSpecies[]: { name, extinctionRGB, scaleHeightMetres, layerCenterMetres, layerThicknessMetres } (for example ozone for Earth)
- groundAlbedoRGB (for the bounce term)
- sunAngularRadiusRadians (for correct limb softening)
- densityAtBottomScalar (for that body's surface pressure)

Per-body starting points (values must be sourced and cited, not invented):

- Earth : ozone absorption required; the sunset colour is the acceptance test.
- Venus : Mie-dominated with a high single-scattering albedo; the surface must be invisible; no Rayleigh blue.
- Mars : dust Mie with a variable optical-depth parameter; blue-ish sunsets come from dust extinction, not from Rayleigh.
- Giants : methane absorption for limb tint; heavy haze.
- Titan : thick organic haze, if ever added.

APPENDIX D – TILE PYRAMID NAMING AND INDEXING

Directory: tiles/L<level>/<x>_<y>.ktx2

- L0 = native resolution of the processed source (do not upscale)
- Each level halves: L1 = L0/2, and so on, until width <= 512.
- x, y are integer tile indices in that level's grid. y increases with latitude going north. Padding of indices is allowed but not assumed by the reader.
- The last tile column in each row duplicates the first tile's first 4 columns (apron) so the longitude seam never shows.
- Tiles are stored uncompressed inside the .zigpack ZIP (they are already KTX2).
- meta/tiles.json lists, per level: width, height, tileCountX, tileCountY, tileSize, apron, and the source file it was derived from.

APPENDIX E – SOURCES & CREDITS SURFACE REQUIREMENTS

The surface is generated at runtime from the installed packs' manifests. It is the product's provenance display: it tells the user where every pixel came from, and it is what lets the owner trace each asset afterwards. For every asset it shows:

- the object name and the asset role (albedo, height, horizon, model, ...),
- the product name and publisher,
- the source URL (displayed and copyable; the app never fetches it - opening a browser is the user's action, and the app has no network permission),
- the exact credit string, verbatim,
- the ground resolution and what the data means (true colour, enhanced, radar, EUV, thermal, simulated, procedural),
- the processing applied by this project (downsampled, re-projected, re-coloured, tiled), so any modification is disclosed.

Plus a short section listing the software the app is built on (Filament, SceneView, KTX-Software, meshoptimizer) with its version. Rules: no logos, no insignia, no endorsement wording, no agency names used as branding, and nothing hand-written - every row comes from a manifest.

APPENDIX F – GLOSSARY

CEILING the finest real resolution available for an object's data. HERO PATCH a regional high-resolution dataset (NAC, HiRISE, Landsat tile). HORIZON MAP per-texel maximum terrain elevation angle by azimuth; drives grazing-sun self-shadowing and ambient occlusion.

ISCO innermost stable circular orbit; the inner edge of the disk. LUT look-up table; precomputed scattering or geodesic data. MATERIAL a Filament material (.mat compiled to .filamat). PACK one Play Asset Delivery asset pack for one object (.zipack). REGOLITH loose rocky surface material (Moon, Mercury, Mars, Lunar). TILE STORE the on-device pyramid + cache + prefetch system. UNIT RADIUS the normalisation where an object's mean radius equals 1.0. VIRTUAL TEXTURE a streaming texture whose resident tiles are managed by the engine.

APPENDIX G – PRIMARY SOURCE URLS PER OBJECT (fetch-time verification list)

Verify each URL at fetch time. If it is unavailable or its terms differ from what is recorded here, STOP and report (rule 2 / rule 3).

SUN

<https://sdo.gsfc.nasa.gov/data/> (AIA/HMI access)
<https://sdo.gsfc.nasa.gov/data/rules.php> (credit request)
http://jsoc.stanford.edu/JSOC_Data_Policy.html (CC0 waiver text)
<https://svs.gsfc.nasa.gov/30362> (Carrington EUV map, labelled use)

MERCURY

https://astrogeology.usgs.gov/search/map/mercury_messenger_mdiss_global_basemap_bdr_166m
https://astrogeology.usgs.gov/search/map/mercury_messenger_mdiss_basemap_loi_global_mosaic_166m
https://astrogeology.usgs.gov/search/map/mercury_messenger_global_dem_665m
https://planetarymaps.usgs.gov/mosaic/Mercury_MESENGER_MDIS_Basemap_MD3Color_Mosaic_Global_665m.tif

VENUS

https://astrogeology.usgs.gov/search/map/venus_magellan_global_c3_mdir_synthetic_color_mosaic_4641m
https://astrogeology.usgs.gov/search/map/venus_magellan_global_topography_4641m
https://astrogeology.usgs.gov/search/map/venus_magellan_global_fresnel_reflectivity_4641m
https://astrogeology.usgs.gov/search/map/venus_magellan_global_meter_scale_slope_4641m

EARTH

<https://visibleearth.nasa.gov/collection/1484/blue-marble>
<https://visibleearth.nasa.gov/images/144898> (Black Marble)
<https://www.usgs.gov/faqs/are-there-any-restrictions-use-or-redistribution-landsat-data>
<https://www.usgs.gov/core-science-systems/nli/landsat/landsat-collection-2> (Landsat access)
<https://www.ncei.noaa.gov/products/etopo-global-relief-model>
<https://nasa-gibs.github.io/gibs-api-docs/> (only for offline-derived clouds)

MOON

https://astrogeology.usgs.gov/search/map/moon_lro_lroc_wac_global_morphology_mosaic_100m
<https://svs.gsfc.nasa.gov/4720> (CGI Moon Kit)
<https://pds-geosciences.wustl.edu/missions/lro/lola.htm>
<https://pds.lroc.im-ldi.com/> (PDS node - the public-domain route)
<https://lroc.im-ldi.com/about/terms> (READ: why the website route is not usable)

MARS

<https://murray-lab.caltech.edu/CTX/>
<https://doi.org/10.17189/m23v-ae12>
https://astrogeology.usgs.gov/search/map/mars_mgs_mola_dem_mosaic_global_463m
https://astrogeology.usgs.gov/search/map/mars_viking_color_global_mosaic_925m
https://astrogeology.usgs.gov/search/map/mars_odyssey_themis_global_mosaic_100m
<https://www.uahirise.org/dtm/> and <https://www.uahirise.org/media/usage.php>

GIANTS

<https://archive.stsci.edu/prepds/opal/> (derive from archived data)
<https://esahubble.org/copyright/> (CC BY 4.0 route)
<https://photojournal.jpl.nasa.gov/catalog/PIA07782> (Cassini Jupiter map)
<https://pds-rings.seti.org/cassini/uvis/index.html> (ring optical depth)
<https://pds-rings.seti.org/ringocc/>

MILKY WAY AND STARS

<https://svs.gsfc.nasa.gov/4851/> (Deep Star Maps; check layer provenance)
<https://www.usno.navy.mil/USNO/astrometry/optical-IR-prod/ucac> (UCAC4)
 Yale Bright Star Catalogue 5th ed. via CDS/VizieR
 IAU Working Group on Star Names list

ISS

<https://science.nasa.gov/resource/international-space-station-3d-model/>
<https://science.nasa.gov/3d-resources/international-space-station-iss-b/>

BLACK HOLE

https://github.com/ebruneton/black_hole_shader (BSD-3-Clause reference)
<https://arxiv.org/abs/2010.08735> (method and performance data)
 James et al. 2015, Class. Quantum Grav. 32, 065001 (documented cinematic choices)
 EHT Collaboration 2019, ApJL 875, L1/L6 (M87* shadow, 42 +/- 3 microarcsec)

ENGINE AND TOOLS

<https://github.com/google/filament>
<https://github.com/google/filament/blob/main/docs/Materials.md.html>
<https://github.com/SceneView/scenview>
<https://github.com/KhronosGroup/KTX-Software> (offline CLI for the asset kit)
<https://github.com/zeux/meshoptimizer>
<https://developer.android.com/guide/playcore/asset-delivery>

ON LICENSING

Not part of this build. The owner clears rights for every asset separately, after delivery, with legal counsel (§2.3 A1). Do not research licence terms, do not add licence fields, do not open a blocker over a licence question. Do keep the source list above accurate and complete: it is the input to that later work.

=====

APPENDIX H – CI, EMULATOR AND DEVICE-LAB SETUP (ready to adapt)

=====

H.1 WORKFLOW 1 – TIER A (no GPU, runs on every push; this is the gating workflow) File: .github/workflows/ci.yml

```
name: ci
on: [push, pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17' }
      - uses: gradle/actions/setup-gradle@v4
      - name: Build
        run: ./gradlew --no-daemon assembleDebug assembleRelease
      - name: Unit tests
        run: ./gradlew --no-daemon test
      - name: Lint
        run: ./gradlew --no-daemon lint
      - name: Provenance gate
        run: ./gradlew --no-daemon checkProvenance
      - name: Scope gate
        run: ./gradlew --no-daemon checkScope
      - name: Offline asset verification
        run: ./gradlew --no-daemon :tools:assetkit:verifyAll
      - name: Release manifest audit
        run: ./tools/ci/check_manifest.sh
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: tier-a-reports, path: '*/build/reports/*' }
```

H.2 WORKFLOW 2 – TIER B (CI emulator with KVM; correctness and interaction only) File: .github/workflows/instrumented.yml

```
name: instrumented
on: [push]
```

```

jobs:
  emulator:
    runs-on: ubuntu-latest
    timeout-minutes: 60
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17' }
      - name: Enable KVM
        run: |
          echo 'KERNEL=="kvm", GROUP="kvm", MODE="0666", OPTIONS+="static_node=kvm" | sudo tee /etc/udev/rules.d/99-kvm4all.rules
          sudo udevadm control --reload-rules
          sudo udevadm trigger --name-match=kvm
      - uses: gradle/actions/setup-gradle@v4
      - name: Instrumented tests on emulator
        uses: reactivecircus/android-emulator-runner@v2
        with:
          api-level: 34
          arch: x86_64
          profile: pixel_6
          emulator-options: -no-window -gpu swiftshader_indirect -noaudio -no-boot-anim -camera-back none
          disable-animations: true
          script: ./gradlew --no-daemon connectedDebugAndroidTest
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: emulator-screenshots, path: '*/screenshots/*' }

```

Notes for the agent:

- The KVM udev step is required on Linux runners; without it the emulator will be unusably slow or will fail to start.
- swiftshader_indirect means software rendering. Never quote timings from this job.
- If the emulator job is flaky, keep it out of the merge requirement for asset-only milestones, but never for engine, UI or tile-store milestones.

H.3 WORKFLOW 3 – TIER C (real devices via Firebase Test Lab, manual, free quota only) File: .github/workflows/device-lab.yml

```

name: device-lab
on: workflow_dispatch
jobs:
  ftl:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17' }
      - uses: google-github-actions/auth@v2
        with: { credentials_json: '${{ secrets.GCP_SA_KEY }}' }
      - uses: google-github-actions/setup-gcloud@v2
      - run: ./gradlew --no-daemon :app:assembleDebug :app:assembleDebugAndroidTest
      - name: Run on physical devices (free quota)
        run: |
          gcloud firebase test android run --type instrumentation --app app/build/outputs/apk/debug/app-debug.apk --test
          app/build/outputs/apk/androidTest/debug/app-debug-androidTest.apk --device model=redfin,version=30 --device model=barbet,version=31
          --timeout 15m --results-dir zig-${{ github.run_id }}

```

Rules:

- Requires a GCP service account in the repository secret GCP_SA_KEY. If it is absent, this workflow cannot run: record that in DEVICE_BACKLOG.md and continue.
- Each run may target up to four devices. Use the free daily quota (see 24.4).

- Never enable a paid plan, never exceed the free quota, never set a billable project. If the quota is exhausted, stop Tier C and note it.

H.4 SCRIPT – TIER D (the human's own phone) File: scripts/device-check.sh (adapt the component name and the extras to the app)

```
#!/usr/bin/env bash
set -euo pipefail
APK="${1:-app/build/outputs/apk/debug/app-debug.apk}"
OBJ="${2:-moon}"
SECS="${3:-60}"
adb install -r "$APK"
adb shell am start -n com.zig.inspector/.MainActivity --es benchmark_object "$OBJ" --es benchmark_seconds "$SECS"
echo "running $OBJ for ${SECS}s; do not touch the device"
sleep $((SECS + 15))
adb exec-out run-as com.zig.inspector cat files/benchmark.csv > "device-$OBJ.csv" || adb pull /sdcard/Android/data/com.zig.inspector/files/benchmark.csv "device-$OBJ.csv"
adb exec-out screencap -p > "device-$OBJ.png"
echo "OK: wrote device-$OBJ.csv and device-$OBJ.png. Send both back."
awk -F, 'NR>1{n++; s+=2; if($2>m)m=$2} END{if(n)printf "frames=%d mean=%.2fms max=%.2fms\n", n, s/n, m}' "device-$OBJ.csv" || true
```

H.5 WHAT THE AGENT MUST NOT DO IN CI

- No emulator performance claims, ever (software rendering).
- No paid cloud device time.
- No secrets in workflow files; use repository secrets only.
- No workflow that downloads third-party assets at build time: assets come from the version-controlled manifests and the offline asset kit, never from a build-time network fetch. The build must work with the network off.

APPENDIX I – LAB SCREEN CARD AND MUSEUM ENTRY POINT (reference sketch)

This appendix is a SKETCH, not a patch. Names, imports, theming and the card composable all come from the real repository. Paste nothing blindly: adapt to the file you actually read in M-1, and delete anything here that does not match it.

I.1 THE CARD CONTRACT (normative parts)

- ONE card, added to the Lab screen's existing card container.
- Title "Space Museum"; one subtitle line; an icon from the app's existing icon set; the existing card component, unmodified.
- A test tag following the file's existing convention (lab_card_space_museum if there is none).
- Single-tap navigation to the museum destination. Nothing else.
- New strings live in the museum module's own res/values/strings.xml, never in the app's existing string resources (§26.2, item 4).
- Existing cards keep their order, tags, text and behaviour: unchanged.

I.2 REFERENCE SKETCH – THE LAB SCREEN EDIT (the whole edit, nothing more)

```
// In the existing Lab screen file. Add ONLY the block marked NEW.
// `LabCard` below stands for whatever card composable the file already uses.
```

```
Column(modifier = Modifier.verticalScroll(rememberScrollState())) {
    // ... existing cards, untouched ...

    // --- NEW ---
    LabCard(
        title = stringResource(R.string.space_museum_title),      // museum module strings
        subtitle = stringResource(R.string.space_museum_subtitle),
        icon = ZigIcons.Museum,                                  // the app's icon set
        modifier = Modifier.testTag("lab_card_space_museum"),
```

```

        onClick = onOpenSpaceMuseum,                // new callback parameter
    )
    // --- END NEW ---
}

```

If the screen's cards are data-driven, add one model entry instead:

```

// --- NEW ---
LabCardModel(
    id = "space_museum",
    titleRes = R.string.space_museum_title,
    subtitleRes = R.string.space_museum_subtitle,
    icon = ZigIcons.Museum,
    testTag = "lab_card_space_museum",
)
// --- END NEW ---

```

If adding the callback parameter changes an existing function signature, record it in docs/integration/CHANGED_FILES.md with the necessity. If it would ripple into other screens, stop and report under S8.

I.3 REFERENCE SKETCH – NAVIGATION (one destination tree)

// In the existing navigation graph file. Add ONLY the museum entries.
 // Adapt to the repo's route style (string routes or type-safe routes).

```

composable(Routes.SpaceMuseumGrid) {
    SpaceMuseumGridScreen(
        onObjectClick = { id -> navController.navigate(Routes.spaceMuseumViewer(id)) },
    )
}
composable(Routes.spaceMuseumViewerPattern) { entry ->
    SpaceMuseumViewerScreen(objectId = entry.arguments?.getString("objectId").orEmpty())
}

```

Rules: the museum grid is the parent of the viewer; system back returns grid ->
 Lab screen; no deep links unless the repo already declares them for every screen;
 no changes to any existing destination's arguments or transitions.

I.4 REFERENCE SKETCH – THE MUSEUM GRID

```

@Composable
fun SpaceMuseumGridScreen(onObjectClick: (String) -> Unit) {
    // Reuse the app's Scaffold, TopAppBar, typography and spacing.
    // ZIG's own theme, not a museum-specific one: it must look native.
    LazyVerticalGrid(columns = GridCells.Adaptive(minSize = 148.dp)) {
        items(ObjectRegistry.all) { spec ->
            MuseumTile(
                name = stringResource(spec.nameRes),
                ceiling = spec.dataCeilingText,        // e.g. "0.5 m/px at Apollo sites"
                onClick = { onObjectClick(spec.id) },
            )
        }
    }
}

```

Requirement: this screen renders and all thirteen tiles appear with ZERO packs installed and no network. Empty-state and missing-pack states are the app's existing pattern, not a new one.

I.5 INTEGRATION TEST CHECKLIST (Tier B emulator is sufficient)

- [] The Lab screen renders with exactly one more card than before.
- [] Every pre-existing card still exists, with the same text and the same test tag, in the same order.
- [] Tapping the new card opens the museum grid.
- [] All thirteen tiles are present and tappable with no packs installed.
- [] Tapping a tile opens the viewer; system back returns to the grid; back again returns to the Lab screen.
- [] ZIG's existing UI tests pass unmodified.
- [] The release manifest still has no INTERNET permission.
- [] An aeroplane-mode run opens the grid and one object with no network calls.

I.6 WHEN THE REPOSITORY DIFFERS FROM THIS SKETCH

It will. That is expected. Write down each difference in docs/decisions/DECISIONS.md with the file and symbol, and implement the version that fits the real repository. The only part of this appendix that is not negotiable is I.1: one card, one destination tree, nothing else moved.

=====

23. CONTINUOUS EXECUTION PROTOCOL

=====

23.1 THE LOOP

implement -> verify -> gate -> (green) commit + tag + PROGRESS.md -> next milestone.

There is no per-milestone approval step. The gate is the approval.

23.2 THE CANONICAL GREEN GATE (applies to every milestone; all items must pass)

- G1 BUILD ./gradlew assembleDebug assembleRelease succeeds with no errors.
- G2 UNIT TESTS ./gradlew test all pass; no skipped tests without a recorded reason.
- G3 STATIC ./gradlew lint no errors (warnings are budgeted, not ignored).
- G4 PROVENANCE the provenance gate (:tools:ci:checkProvenance) passes and prints the asset count it checked. Zero assets checked is a FAILURE, not a pass. (There is no licence check: licensing is out of scope.)
- G5 ASSETS the offline asset verification suite (24.2) passes for every pack built so far.
- G6 CI the pushed commit's GitHub workflow is green: Tier A always, and Tier B (24.3) whenever the milestone touched engine, UI or asset code. If the push could not be made because GitHub is throttling (§28.4), record CONDITIONAL-CI in PROGRESS.md with the pending commit SHA and continue: an unreachable GitHub never blocks a milestone.
- G7 DOCS DECISIONS.md updated for every non-obvious choice; PROGRESS.md updated with status, commit SHA and the evidence path.
- G8 SCAN no INTERNET permission in the release manifest, no HTTP client and no WebView reference anywhere in the source tree.
- G9 EVIDENCE docs/verification/M<N>/ contains the command outputs, test reports and any screenshots or CSVs the DoD requires.
- G10 SCOPE the scope gate (:tools:ci:checkScope) passes: every changed pre-existing file is accounted for in docs/integration/CHANGED_FILES.md with a necessity that survives the §26.2 test, and ZIG's existing tests still pass unmodified. The report reproduces the changed-path list rather than asserting that it was clean.

23.2b PUSH BUDGET INSIDE THE GATE

G6 does not mean "push until green". It means: push ONCE per milestone, at the

gate, and read the result. If the result is red, fix locally, then push again - that second push is allowed as part of the same milestone, subject to the 20-minute spacing of §28.2. Never push work-in-progress to trigger CI.

23.3 WHAT DOES NOT BLOCK ADVANCEMENT

Device-only evidence: real-GPU frame times, thermal behaviour, driver-specific artefacts, and pixel certification on real hardware. These are appended to docs/verification/DEVICE_BACKLOG.md with (a) what to measure, (b) the exact command that produces it, and (c) which DoD item it satisfies. The milestone is still complete; the backlog entry is the debt, and it must be closed before the final acceptance checklist (section 22) is signed off.

GitHub reachability: a throttled push, an auth-looking failure, or an unreachable CI is never a blocker (§28.4). Keep building locally; the push happens at the next opportunity, and G6 becomes CONDITIONAL-CI in the meantime.

23.4 HARD STOP CONDITIONS (the only reasons to stop and ask the human)

- S1 An asset or dataset required by the milestone cannot be found, or the source appears to be a different product than this brief describes (A3). Report it and use the next entry in the §27.4 ranking.
- S2 A pinned toolchain component (Filament, for example) cannot be introduced into the existing repository at any version compatible with its current setup, and no compliant alternative exists.
- S3 A library or engine API named in this brief does not exist in the pinned version and no compliant equivalent can be implemented.
- S4 Two requirements in this brief contradict each other and the resolution is a product decision.
- S5 A gate has failed twice after genuine attempts to fix it and the cause is outside your control (a CI outage, an exhausted quota, a missing secret - and for anything GitHub-related, only after the four retries of §28.4).
- S6 The work would require changing scope: adding an object, adding a network feature, dropping a labelling requirement, or shipping an unverifiable claim.
- S7 The free Firebase Test Lab quota is exhausted and Tier C evidence is required for a milestone you cannot otherwise complete (see 24.4).
- S8 The feature cannot be added without a substantial rework of existing ZIG code - a redesign, a refactor, a rework of an existing flow (§26.7, §4.5). Do not rework the existing code to make the feature fit.

When stopping, state: the milestone, the exact blocker, the evidence you gathered, what you tried, and the options you see. Do not guess and do not substitute.

23.5 PROGRESS TRACKING

PROGRESS.md is the single source of truth for status. One row per milestone:
 | Milestone | Status | Gate | Commit | Date | Evidence path | Deferred items |
 Status vocabulary: GREEN (gate + DoD pass), CONDITIONAL (DoD passes but a device-backlog item is outstanding), BLOCKED (a hard stop condition applies).
 Update it in the same commit that completes the milestone, and tag that commit m<N>-green.

23.6 GITHUB WORKING AGREEMENT (the short version; section 28 is binding)

- One branch per milestone: m03-tile-store. Never commit milestone work directly to main. Never force-push anything, ever.
- Conventional commits (feat:, fix:, chore:, docs:, test:, perf:). Commit locally as often as you like; pushes are rationed (§28.2): ONE PUSH PER COMPLETED MILESTONE, never two within 20 minutes, never more than six in an hour.
- Open a pull request per milestone; the PR description is the section 20 report; self-review it against the DoD checklist; merge it before starting the next milestone's branch. One open PR at a time.

- Tag completed milestones m0-green, m1-green, ... and push branch and tag in the same push.
- If a milestone must be revisited, open a new branch fix-m<N>-<topic>; never rewrite history that has been tagged.
- Keep docs/verification/ and manifests/ in version control. Keep assets-src/ and assets-built/ out of it (git-ignored) and reference packs by SHA-256.
- Never modify an existing CI workflow: add jobs, or add new workflow files.

24. VERIFICATION WITHOUT A LOCAL DEVICE

24.1 THE SITUATION, STATED PLAINLY

The implementing agent has a GitHub repository but no emulator and no phone. Most of this project is verifiable anyway, because the renderer is a thin layer over data and mathematics. Four verification tiers are defined; only the last two need real hardware, and neither is allowed to block development.

24.2 TIER A – HOST AND CI, NO GPU (always required, always gating)

A1 JVM unit tests: camera maths, sun-direction function, unit-radius conversion, registry values, tile level selection, catalogue samples, black-hole physics.

A2 Builds: debug and release variants; release manifest audit (no INTERNET).

A3 Provenance gate over every manifest, and the scope gate over the diff.

A4 THE OFFLINE ASSET VERIFICATION SUITE (the key instrument; implement it in M2 and extend it at every milestone that adds assets). It validates built packs without a GPU:

- KTX2 validity: magic, format, dimensions, mip count, transfer function flag as declared in the manifest.
- Colour-space audit: albedo maps are sRGB; normals, height, masks and horizon maps are linear. A mismatch fails the gate.
- Mip sanity: per-level mean and variance behave monotonically; no two levels are byte-identical; mip level count matches the declared size.
- Tile seam continuity: the mean absolute difference across every pair of adjacent tile edges is below a threshold; the longitude seam tile is duplicated, not wrapped implicitly.
- Pole check: distortion of a synthetic circle near the pole is below a threshold for the chosen mapping.
- Horizon map correctness: run the generator on a synthetic cone DEM and assert the horizon angles match the analytic values within a tolerance.
- Normal map correctness: run on a synthetic ramp of known slope and assert the decoded normals match the analytic normal within a tolerance.
- LUT verification: the black-hole tables and atmosphere LUTs are compared against direct numerical integration, printing the maximum error.
- Catalogue validation: for a fixed list of named stars, assert position, magnitude and colour index against published values.

This suite is what makes correctness assertable without hardware. It runs in CI on every push and its output is committed under docs/verification/.

24.3 TIER B – CI EMULATOR (GitHub Actions, ubuntu runner with KVM)

Available and usable: GitHub-hosted Linux runners expose KVM, and the Android emulator runs hardware-accelerated once the KVM udev permissions are enabled (see Appendix H for a working workflow). Recent practitioner reports indicate macOS runners no longer reliably support the nested virtualisation this needs, so use ubuntu-latest.

IMPORTANT LIMITATION: the emulator renders with SwiftShader (software GL). It is a correctness environment, not a performance environment, and its pixels do not match a real GPU.

USE IT FOR: app launch, opening all thirteen objects without crashing, Compose interaction tests (controls, toggles, HUD text, credits list), resource-leak

assertions across 100 object switches, screenshot capture of every object as smoke evidence, and instrumented tests of the tile store's state machine. DO NOT USE IT FOR: frame rate, thermals, driver artefacts, or pixel-exact golden certification. Emulator goldens are recorded as SMOKE with an inflated tolerance and are never quoted as evidence of visual quality.

24.4 TIER C – REAL DEVICES VIA FIREBASE TEST LAB (free quota, no billing needed)

Firebase Test Lab's free tier provides 10 virtual and 5 physical device test runs per day on the Spark plan, each run targeting up to four devices, with the quota refreshing daily; no payment method is required for that quota. Over the quota, virtual devices bill at about \$1 per device-hour and physical devices at about \$5 per device-hour - the agent must NEVER incur charges.

USE IT FOR: the benchmark mode on three to four physical devices per batch (frame times, thermal behaviour), golden screenshot capture on real GPUs, and the black-hole and displacement checks that depend on real shader compilation. RULES: trigger it manually (workflow_dispatch), at most once per milestone, only on the free quota, and stop immediately if the quota is exhausted (hard stop S7 if Tier C evidence is then required). Record device model, GPU, Android version and the CSV under docs/verification/M<N>/device-lab/.

PREREQUISITE: the human supplies a GCP service-account secret. If that secret is absent, Tier C is unavailable - the agent must say so explicitly, queue the items, and continue. It is never a reason to stop the whole plan.

24.5 TIER D – THE HUMAN'S OWN DEVICE (the device validation kit)

Ship scripts/device-check.sh with every milestone that adds an object. It installs the built APK over adb, runs the benchmark for a named object for a fixed number of seconds, pulls the CSV and a screenshot, and prints a one-line summary with the p50/p95 frame times. Hand it to the human at each milestone so real-hardware evidence accumulates in parallel with development instead of at the end.

The agent must keep this script working: changing an adb target, an extra name or a file path in the app requires updating the script in the same commit.

24.6 WHAT CANNOT BE VERIFIED WITHOUT A REAL DEVICE - EVER

- Sustained frame rate and thermal behaviour over ten minutes.
- GPU-specific artefacts: tile seams under anisotropic filtering on a particular driver, half-float blending differences, long-shader compilation behaviour on Mali-class GPUs.
- Final visual judgement against the product standard.

These items are recorded in DEVICE_BACKLOG.md, never asserted as passing, and are closed only by Tier C or Tier D evidence.

24.7 PROHIBITED CLAIMS (reviewed at every gate)

Never write "60 fps verified", "performance measured", "golden images pass", or "no artefacts" unless the evidence came from Tier C or Tier D. Otherwise write TARGET, ESTIMATE, or "pending device validation". A report that overstates a measurement fails the gate as surely as a failing test.

25. HANDOVER REPORT (produced when the plan completes or a hard stop occurs)

1. STATUS: milestones green / conditional / blocked, with tags.
2. WHAT EXISTS: modules, packs built, objects shipped, tools implemented.
3. GATE SUMMARY: the last full green run, its commit SHA, and the CI run URL.
4. DEVICE BACKLOG: every unresolved device-only item, with the command to run it.
5. SOURCES: the asset count, and the complete source index (product, publisher,

- URL) for every shipped asset, so the owner can clear rights afterwards. Note anything that was selected but could not be obtained, and what was used instead.
- 6. DEVIATIONS: the full DECISIONS.md index, one line each.
- 7. HOW TO BUILD AND REBUILD: exact commands for the app and for every pack (docs/pipeline.md is authoritative; repeat the entry points here).
- 8. THE NEXT THREE THINGS a human should do, in priority order.

26. INTEGRATION INTO THE EXISTING ZIG APP

26.1 WHAT SPACE MUSEUM IS, FROM THE USER'S POINT OF VIEW

- ZIG gains one new card in its existing Lab screen, labelled "Space Museum", with a one-line subtitle. Nothing else in the Lab screen changes.
- Tapping that card opens the Space Museum: a screen of thirteen object tiles, presented in the app's own visual language.
- Tapping a tile opens the viewer: one object, rendered locally, to be orbited, zoomed and inspected. System back returns to the museum grid, then to the Lab screen.
- The feature is fully offline. It must open and render with the device in aeroplane mode and with no packs installed (base assets only), for all thirteen objects.
- The feature is not a mode of any existing screen. It is a new destination, reachable from the Lab screen and from nowhere else.

26.2 TOUCH NOTHING IRRELEVANT (THE RULE)

This brief cannot tell you which files matter: you know the repository, and its author does not. So the instruction is not a list of forbidden files - it is one test, applied to every edit, everywhere:

DOES SPACE MUSEUM FAIL TO WORK WITHOUT THIS CHANGE?

If the answer is not an unqualified yes, do not make the change. What that means in practice, as examples rather than an exhaustive list:

- Existing calculations - orbital, ISS tracking, positional, timing, units, precision - are not modified, refactored, renamed, optimised or "cleaned up".
- Existing screens, navigation, layouts and flows are not redesigned. The smallest edit that makes Space Museum reachable is the correct edit.
- Existing models, string resources, drawables, assets, dependencies, build configuration and CI jobs are left exactly as they are. Anything a new module needs is added inside that new module.
- Existing tests are never edited to accommodate the new feature. If an existing test fails, the new feature is wrong.
- No reformatting, no import reordering, no comment or whitespace churn, no version bumps, no dependency alignment, no "while I was here" improvements.

None of this is about the author's taste. Every unnecessary edit is a regression risk in an app that already works and already has users.

26.3 HOW TO ADD WITHOUT DISTURBING

- Prefer NEW code to MODIFIED code, and additions to rewrites: a new function, a new parameter with a default value, a new entry in an existing list.
- To make the feature reachable from an existing screen, make the smallest edit that achieves it, in the idiom that file already uses.
- A new dependency is added for the new modules only; nothing existing is bumped, removed or realigned.

- New strings, drawables and icons live in the new modules' own resources.
- Keep every pre-existing file's diff as small as the task allows, and make each one traceable to a necessity in docs/integration/CHANGED_FILES.md.
- Nothing in the new feature may be load-bearing for the rest of ZIG: no existing code may come to depend on the museum's modules, packs or manifests.

26.4 RECONNAISSANCE (M-1, read-only)

Produce, with real paths and real symbol names, and with nothing invented:

- docs/integration/INVENTORY.md: modules, package roots, SDK levels, actual AGP/Kotlin/Compose/JDK versions, DI approach, navigation approach, theming, resource conventions, test setup, CI jobs.
- docs/SPEC_MUSEUM.md: the feature as it will exist in THIS repository - module names, package names, screen structure, where it will be reachable from, and the minimum edits that will make it reachable.
- docs/integration/CHANGED_FILES.md: every pre-existing file you expect to touch, one line each, with the necessity.
- docs/integration/COMPONENT_REUSE.md: the existing components, typography, colours, spacing and icons the card and the museum grid will reuse.
- docs/integration/ENVIRONMENT.md: the build environment the feature needs and how it differs from section 3.
- docs/SOURCES.md: the source index (A2).

No production code is written in M-1.

26.5 THE LAB SCREEN CARD (UI contract, to be adapted)

PLACEMENT	Inside the same card container the Lab screen already uses, in the order the file already expresses. If that order is semantic (grouped by purpose), the museum card joins the exploration/education group; record the reason in DECISIONS.md.
CONTENT	Title "Space Museum"; one subtitle line in the app's established subtitle style; an icon from the app's existing icon set (never a new icon dependency); the existing card component, unmodified.
SEMANTICS	A test tag following the file's existing convention, for example lab_card_space_museum. Every existing card must remain addressable by its existing test tag, and existing UI tests must pass unchanged.
BEHAVIOUR	Single tap navigates to the museum grid. No long-press menu, no swipe action, no badge, no tooltip, no animation added to the screen.
RESTRAINT	One card, one destination tree. No changes to the Lab screen's header, layout, scroll behaviour or the ordering of existing cards.
WARNING	If the screen cannot accept a card without a redesign - a closed enum, a hand-tuned layout with no room, a hard-coded list elsewhere - stop and report under S8. Do not redesign the screen.

26.6 PROOF OF MINIMUM DIFF (required evidence, every milestone)

1. docs/integration/CHANGED_FILES.md is updated: every PRE-EXISTING file the milestone touched, one line each, stating why the change is REQUIRED. New files need no entry.
2. The §26.2 test is applied to each entry. An entry whose reason does not survive it is not a deviation to be justified - it is a change to be undone before the gate.
3. ZIG's existing unit and instrumentation tests run unmodified and pass.
4. The scope gate (§16.4) prints the changed-path list; the report reproduces it rather than asserting that it was clean.
5. A check (scripted in :tools:ci) that no existing module has gained a dependency on the museum modules, other than :app.
6. A before/after screenshot of the Lab screen showing the new card and nothing else moved (the Tier B emulator is sufficient).

7. Every existing card, screen and control still behaves identically.

26.7 HOW TO DECIDE WHEN YOU ARE UNSURE

Apply the test of §26.2. If the honest answer is "it is not strictly necessary, but it is cleaner / more modern / the linter wants it / it was already wrong / it will help later", do not make the change. Write the observation in docs/decisions/DECISIONS.md under "found, not touched", with the file and line, and move on.

The existing app wins, always. If a pattern in ZIG is not what you would choose, follow it anyway: consistency with the host application outranks your preference. If the feature cannot be built without a substantial rework of existing code - a redesign, a refactor, a rework of a flow - stop and report under S8. That is not a failure; it is the correct outcome of recognising a boundary.

27. ASSET SELECTION: THE BEST AVAILABLE, PER OBJECT

27.1 THE RULE

Choose the best asset that exists for each object. Not the cheapest, not the easiest to obtain, not the one with the simplest terms - the best one. Quality decides, and section 27.4 records the selection.

Licensing, attribution rights, permissions and registries are NOT part of this build. The owner handles them himself, with a law professional, after the feature is finished and he is satisfied with it (A1). Make no licence decisions, keep no licence registry, open no blocker over a licence question, and add no licence logic to the app or to CI (§16.1).

What you do owe the owner is TRACEABILITY (A2): for every asset that ships, the product name, the publisher, the credit string and the URL it came from, in the pack manifest and in docs/SOURCES.md. That is the input to his later work, and it is the only part of this subject that is yours.

27.2 WHAT STILL BINDS

1. Never invent an asset, dataset, product name or URL. Everything must exist and come from where the record says it comes from (A3).
2. Assets ship inside the app. No runtime fetching, no streaming, no WebView, no remote rendering - the release build has no network permission at all.
3. Write the code yourself. Third-party source code is never pasted or vendored into this repository; external code enters only as a declared, pinned dependency (A4). A shader is code.
4. Scientific honesty is unaffected by where an asset came from: true colour, enhanced, radar, thermal, simulated and procedural-beyond-data are labelled as what they are (§5.9), and the data ceiling governs what may be shown.
5. No agency logos, insignia, mission patches or endorsement wording. That is a product-presentation rule, not a licence question, and it applies regardless.
6. If the asset you selected turns out to be a different product than described, or cannot be obtained at all, report it (S1) and use the next entry in the ranking. Never substitute silently.

27.3 HOW TO BREAK A TIE (in this order, highest first)

1. Measured data beats rendered art, always.
2. Higher real ground resolution beats lower: compare metres per texel, not the megapixels of the delivered file.
3. A calibrated, documented product beats an image of unknown processing.
4. A published product name beats "a texture I found": provenance must be citable.

5. Closer to true colour beats more saturated.
6. Only then consider aesthetics. When two candidates tie on 1-5, prefer the one that requires less invention at the far end of the zoom.

27.4 THE SELECTION, PER OBJECT (best available, in ranking order)

Read this as the default plan. Where reality contradicts it - a product has moved, a better one has appeared, an archive is down - use the next best, record what you actually used, and say so in the report.

Object	Selected (best available)	Why it wins, and what to record
Sun	SDO/AIA + HMI full-disk series at native 4096x4096 (JSOC), plus the Carrington EUV map (NASA SVS item 30362)	Real instrument data at instrument resolution; nothing anywhere exceeds it. Record the series and the credit line requested by the data provider
Mercury	MESSENGER MDIS Basemap BDR 166 m global (92160x46080) + LOI + MD3 colour + Global DEM 665 m	The highest-resolution global map of Mercury that exists, at any price. Record the USGS product pages actually downloaded
Venus	Magellan C3-MDIR radar + F-MIDR + GTDR topography, plus a VIRTIS/VMC colour pass	Only global surface dataset in existence; the colour pass replaces the fake-yellow look with measured spectral behaviour. Record both the PDS products and the ESA colour product
Earth	Blue Marble NG 21600x10800 + Landsat-8/9 15 m mosaic + Copernicus/SRTM DEM; a high-resolution commercial or specialist basemap may replace the albedo where it is genuinely better	Global consistency at 15 m with real relief; the upgrade is worth it only where the replacement beats Landsat on resolution for the regions users zoom to
Moon	LROC WAC 100 m morphology + LOLA 59 m height field + LROC NAC imagery and DTMs at 0.5 m for hero sites	0.5 m/px is the finest planetary surface data in existence, and it is the showcase of this object. Record the NAC product ids per hero site
Mars	Murray Lab CTX 5 m global mosaic + MOLA/HRSC DEM + HiRISE imagery and DTMs at 0.25-0.5 m for hero sites	5 m globally with sub-metre detail where it matters; no competitor product approaches it
Jupiter	OPAL calibrated annual global maps (0.1 deg/px) + JunoCam frames + Cassini PIA07782 for continuity	The only global sampling of Jupiter at this scale; JunoCam adds genuinely new detail at the pole and in the belts
Saturn	OPAL/Cassini ISS global maps + ring optical-depth profiles from the PDS ring node (COUVIS_8001, CORSS_8001)	Ring structure is measured, not drawn: the profiles are what make the rings honest. Record the ring product ids
Uranus	OPAL maps + Irwin et al. 2024 corrected colour + Voyager 2 archive mosaics	Voyager 2 is the only close-range data that exists; OPAL supplies the modern global map
Neptune	OPAL maps + Voyager 2 1989 mosaics + Irwin et al. corrected colour	One flyby, one epoch, no substitute; the corrected colour fixes the famously over-saturated Voyager palette
Milky Way	ESO/S. Brunier gigapixel Milky Way panorama + DSS2 imagery + Gaia DR3 star layer + UCAC4/BSC5 bright stars	The only real gigapixel all-sky image, plus the best astrometric catalogue in existence: this raises the star field from thousands of true positions to millions
ISS	NASA VTAD ISS_stationary.glb (44.5 MB, 247547 triangles, 28 PBR materials) + the other official configurations	Official geometry with PBR materials; no purchased model is more accurate. Record the package version you downloaded
Black hole	EHT M87 (2019) and Sgr A (2022) released data as the appearance reference, plus the Bruneton 2020 precomputed-deflection method implemented here	The EHT releases are measurements; they fix the shadow diameter and the lensing scale the renderer must reproduce. Record the release you used, and label the cinematic mode as cinematic

Every selected row above is also the answer to "what do I record?": the product, the publisher, the URL and the credit string, in the manifest and in docs/SOURCES.md. That list is the deliverable the owner needs from this section.

27.5 WHAT A BIGGER BUDGET BUYS, AND WHAT IT CANNOT

Worth the effort, because the gain is real:

- Milky Way: Gaia DR3, DSS2 and the gigapixel panorama replace a medium-resolution deep map. This is the single largest visible improvement in the whole feature.
- Jupiter, Saturn, Uranus, Neptune: modern, uniform, calibrated global maps where the older free-only route forced mosaicking and epoch mismatch.
- Moon: NAC products at 0.5 m/px for the hero sites, and honest polar coverage.
- Earth: a better albedo wherever the replacement genuinely beats Landsat.
- Black hole: EHT geometry as the ground truth for the ring.
- Sun: the full AIA series and the Carrington map, so the EUV mode is real data.

Not worth the effort, because nothing exists to buy:

- Mercury, Venus, Mars. MESSENGER MDIS, Magellan and the CTX mosaic are already the best data in existence. Record them and move on.
- Any "planet texture pack" sold as a product: nearly always a downsampled derivative of exactly the archive data above, with no provenance chain.
- Anything promising surface detail beyond the data ceiling. The ceiling is a property of the instruments, not of the budget, and no purchase raises it.

28. GITHUB DISCIPLINE, RATE LIMITS, AND THE FALSE DISCONNECT

28.1 WHY THIS SECTION EXISTS

An agent that pushes on every task, opens branches freely, polls the API in loops or re-authenticates when it sees an error will trip abuse detection: first throttling, then temporary authentication failures, then, if it keeps going, account restrictions. A banned account ends the delivery pipeline, so push discipline is a functional requirement of this project, not a courtesy. The second half of this section covers a specific, expected false alarm: GitHub looking "disconnected" when it is merely throttling.

28.2 THE PUSH BUDGET (hard numbers, not guidance)

- ONE PUSH PER COMPLETED MILESTONE. Roughly fifteen pushes for the whole plan, not hundreds.
- Never two pushes within 20 minutes. Never more than six pushes in any hour.
- Commit locally as often as you like: commits inside a single push cost nothing.
- At most one open pull request at a time. Close (merge) it before opening the next milestone's PR.
- No polling loops. Check a CI run at most once every five minutes, at most four times per run, preferring a single `gh run view --json status,conclusion` at the end. Never `watch` in a tight loop.
- Never trigger workflows with work-in-progress pushes. If a manual run is genuinely required, use one `workflow\_dispatch`, once per milestone.
- Push the milestone branch and its tag in the same push.
- Never force-push. Never rewrite published history. Delete merged branches.

28.3 THE LOCAL-FIRST WORKFLOW

- Work locally; produce the gate evidence locally (it lives in docs/verification/M<N>/ anyway, so it must exist before the push).
- Push once when the milestone's gate is green locally. CI then confirms what the gate already established. A red CI result is a build problem, not a disaster: fix it, then push again - subject to the spacing rule.
- Nothing in the project depends on GitHub being reachable at any given moment. If it is unreachable, continue with the next milestone's local work and push at the next milestone boundary.

28.4 THE FALSE-DISCONNECT PROTOCOL (what to do when it looks broken)

Symptoms that are almost always throttling rather than a lost repository: "Authentication failed"; "could not read Username"; "Repository not found"; 403; 429; "fatal: unable to access ... The requested URL returned error"; a push that hangs; access that worked minutes ago and fails now.

DO:

1. Confirm the remote is still there: `git remote -v`. Then ONE read-only reachability test: `git ls-remote <remote>` (not once per minute).
2. WAIT 15 MINUTES. Not fifteen seconds. Do not retry in a loop.

3. Retry the identical command, unchanged.
4. Repeat the 15-minute wait and retry, up to four attempts in total (about an hour).
5. Keep working while waiting: implement the next milestone, write tests, build and verify packs locally. Waiting time is not lost time.
6. Record each event in PROGRESS.md: timestamp, symptom, attempt number, outcome.

DO NOT:

- Re-authenticate, rotate or delete credentials, edit git config, or change the remote URL.
- Re-clone the repository or re-initialise git.
- Create a new repository, fork, mirror or second remote.
- Force-push, or push from a different account.
- Assume work is lost. It is committed locally; nothing is lost.
- Declare a blocker before the fourth attempt. After four failures the sync is blocked - the build is not. Continue building; note it in PROGRESS.md.

28.5 ACTIONS THAT WILL GET THE ACCOUNT BANNED (never, under any circumstance)

Action	Why it is dangerous	Do this instead
Push after every file or task	Bursts look like abuse automation	Batch to one push per milestone
Retry a failed push in a tight loop	Escalates throttling to restriction	Wait 15 minutes, then retry
Force-push to a shared or tagged branch	Destroys history, flags the account	Open a fix branch and merge
Create a branch or PR per task	Request volume is the trigger	One branch and one PR per milestone
Poll the API or Actions in a loop	API abuse detection	One check per 5 minutes, max four
Re-authenticate on every error	Credential churn is a fraud signal	Wait; the token is fine
Delete and recreate the repo or remote	Looks like account takeover	Leave it alone; keep building
Mass-create issues, comments or labels	Spam classification	Write one clear report per milestone
Trigger workflows with WIP commits	Wasted minutes also count against you	Use one workflow_dispatch per milestone

28.6 SIGNALS, DIAGNOSIS, ACTION

Signal	Almost always means	Action
403 with a rate-limit body	Secondary rate limit	Wait 15 minutes; retry
429	Too many requests	Wait 15 minutes; retry
"Authentication failed" after a working session	Throttled auth, not a bad token	Wait 15 minutes; retry unchanged
"Repository not found" on a repo you just used	Throttled or expired session cache	Wait; verify with git ls-remote
Push hangs then times out	Network or throttling	Wait; retry once, then continue locally
"Another git process seems to be running"	A lock file, not GitHub	Remove the stale index.lock and retry
CI red, unrelated job	Your change or a flake	Fix or re-run once; never loop

28.7 THE GATE UNDER RATE LIMITS, AND WHAT TO REPORT

- Gate item G6 becomes: CI green if the push succeeded; otherwise CONDITIONAL-CI, recorded in PROGRESS.md with the pending commit SHA, the reason and the next retry time. A milestone is NOT blocked by an unreachable GitHub.
- PROGRESS.md gains one line per milestone:
 - | Sync | pushes | retries | last attempt | CI state |
- The section 20 report's item 9 (SYNC) carries the same information, and item 7 (blockers) must separate build blockers from sync delays: they are different kinds of problem and conflating them makes the report useless.
- Over the whole plan, expect roughly 15 pushes and zero force-pushes. If the count is wildly higher, the agent has misread this section.

=====

END OF DOCUMENT

=====